

实验概述

1. 实验目的

- （1）理解单周期CPU的结构及工作原理；
- （2）熟悉miniRV或miniLA指令集；
- （3）掌握单周期CPU的设计与实现方法。

2. 实验内容

本实验要求在模板工程的基础上，设计实现支持miniRV或miniLA指令集的单周期CPU。

miniRV

miniRV指令集如表0-1所示。其中，浅蓝色背景的是A组指令，浅橙色背景的是B组指令，绿色字体的8条指令是模板工程已经实现的示例指令。

表0-1 miniRV指令概览表

示例指令（模板工程已实现）

指令功能	指令	类型	指令功能	指令	类型
移位运算指令	SLLI	I型	Load指令	LW	I型
算术运算指令	ADDI	I型	直接跳转指令	JAL	J型
	LUI	U型	条件分支指令	BEQ	B型
逻辑运算指令	ORI	I型		BNE	B型

A组（18条）

B组（18条）

指令功能	指令	类型	指令功能	指令	类型
移位运算指令	SLL	R型	算术运算指令	MUL	R型
	SRL	R型		MULH	R型
	SRLI	I型		MULHU	R型
	SRA	R型		DIV	R型
	SRAI	I型		DIVU	R型
算术运算指令	ADD	R型		REM	R型
	SUB	R型		REMU	R型
	AUIPC	U型	比较运算指令	SLT	R型
逻辑运算指令	XOR	R型		SLTI	I型
	XORI	I型		SLTU	R型
Load指令 （读访存）	LB	I型		SLTIU	I型
	LBU	I型	逻辑运算指令	OR	R型
	LH	I型		AND	R型
	LHU	I型		ANDI	I型
Store指令 （写访存）	SW	S型	条件分支指令	BLT	B型
	SB	S型		BGE	B型
	SH	S型		BLTU	B型
直接跳转指令	JALR	I型		BGEU	B型



miniLA指令集如表0-1A所示。其中，浅蓝色背景的是A组指令，浅橙色背景的是B组指令，绿色字体的8条指令是模板工程已经实现的示例指令。

表0-1A miniLA指令概览表

示例指令（模板工程已实现）

指令功能	指令	类型	指令功能	指令	类型
移位运算指令	SLLI.W	2RI5型	Load指令	LD.W	2RI12型
算术运算指令	ADDI.W	2RI12型	直接跳转指令	B	I26型
	LU12I.W	1RI20型	条件分支指令	BEQ	2RI16型
逻辑运算指令	ORI	2RI12型		BNE	2RI16型

A组（19条）

B组（18条）

指令功能	指令	类型	指令功能	指令	类型
移位运算指令	SLL.W	3R型	算术运算指令	MUL.W	3R型
	SRL.W	3R型		MULH.W	3R型
	SRLI.W	2RI5型		MULH.WU	3R型
	SRA.W	3R型		DIV.W	3R型
	SRAI.W	2RI5型		DIV.WU	3R型
算术运算指令	ADD.W	3R型		MOD.W	3R型
	SUB.W	3R型		MOD.WU	3R型
逻辑运算指令	PCADDU12I	1RI20型	比较运算指令	SLT	3R型
	XOR	3R型		SLTI	2RI12型
Load指令 （读访存）	XORI	2RI12型		SLTU	3R型
	LD.B	2RI12型		SLTUI	2RI12型
	LD.BU	2RI12型	逻辑运算指令	OR	3R型
	LD.H	2RI12型		AND	3R型
Store指令 （写访存）	LD.HU	2RI12型		ANDI	2RI12型
	ST.W	2RI12型	条件分支指令	BLT	2RI16型
	ST.B	2RI12型		BGE	2RI16型
直接跳转指令	ST.H	2RI12型		BLTU	2RI16型
	JIRL	2RI16型		BGEU	2RI16型
	BL	I26型			

本实验的具体要求如下：

- （1）小组成员需各自挑选一组指令，并完成对应的数据通路表、控制信号表；
- （2）与组员讨论协作，共同完成完整单周期CPU的数据通路图绘制；
- （3）小组成员各自完成A、B组指令的单周期CPU设计与实现；
- （4）小组成员各自调试，通过A、B组指令的单周期CPU的Basic Trace测试；
- （5）与组员讨论协作，共同把代码整合成完整的单周期CPU，并通过Basic Trace测试。

熟悉代码及模板工程

本课程提供了已实现8条指令的单周期CPU demo工程，并且这8条指令的数据通路和控制信号均已包含在数据通路表的模板当中。

demo工程已实现的8条参考指令

miniRV示例工程已实现的指令有： `addi`、`ori`、`slli`、`lui`、`lw`、`beq`、`bne`、`jal`。

miniLA示例工程已实现的指令有： `addi.w`、`ori`、`slli.w`、`lui2i.w`、`ld.w`、`beq`、`bne`、`b`。

在开始设计你的CPU之前，请使用demo工程运行功能仿真，结合仿真波形，对照实验原理、数据通路表及示例代码，分析CPU架构及工作原理，理解指令在CPU中的执行过程，体会不同指令执行时数据通路的变化。

demo工程同时可作为模板工程，后续可在此基础上进行开发。

1. 下载demo工程并运行仿真

点击实验指导书首页的课程材料下载链接，根据需要选择下载对应的demo工程。

有4个demo工程，我要下载哪个？



图1-1 根据实际情况，选择其中一个demo工程

首先，选择指令集是miniRV或miniLA，然后再根据上课的实验室进行选取 —— 在T2210、T2507上课的同学，请选择文件名带有“`ego1`”后缀的工程；而在T2506、T2612上课的同学，则选择文件名带有“`minisys`”后缀的工程。

在Vivado界面左侧点击运行功能仿真，如图1-2所示。等待一段时间后，出现仿真波形窗口。

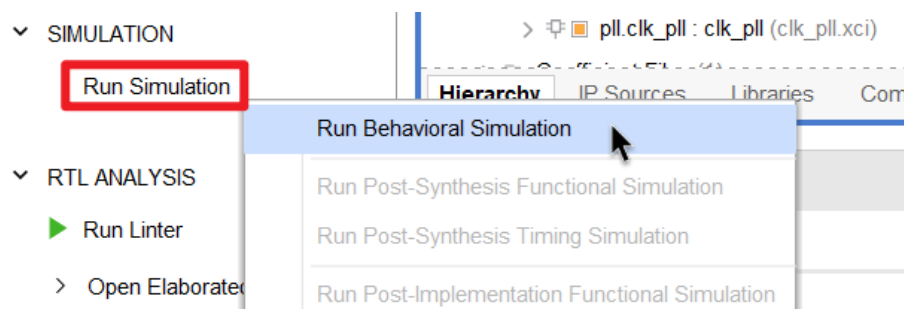


图1-2 运行功能仿真

点击Run All按钮，如图1-3所示，Testbench将通过测试并停止。

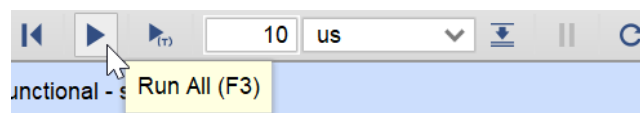


图1-3 继续仿真直至结束或报错停止

此时的仿真波形是CPU运行`lw.dump`测试程序（miniLA则是`ld.w.dump`）所产生的。

仿真波形窗口默认包含了复位、时钟、外设信号，以及CPU的取指信号和数据访存信号，各信号的解释和说明如图1-4所示。



图1-4 波形配置文件 soc_simple_tb.wcfg 的信号说明

外设信号的波形为什么是高阻 Z?

demo工程尚未实现外设的I/O接口电路，无法读写外设数据，相关信号也尚未连接，因此仿真波形上部分外设信号呈现高阻状态，这是正常的。

2. CPU模块的接口信号

由图1-4可知，CPU模块对外有两组接口信号，分别用于取指令和读写主存的数据，这两组接口信号的定义分别如表1-1、表1-2所示。

表1-1 CPU对外的取指接口信号

信号	位宽	属性	功能描述

ifetch_req	1	输出	取指请求信号：高电平表示CPU需要取指
ifetch_addr	32	输出	取指的地址（字节地址）
ifetch_vali d	1	输入	指令存储器返回的指令的有效信号 有效一个时钟表示返回一条指令
ifetch_inst	32	输入	指令存储器返回的指令机器码

取指时序解读

此处以miniRV的demo工程为例，介绍CPU的取指时序。在仿真波形上任意选取一段波形，如图1-5所示。

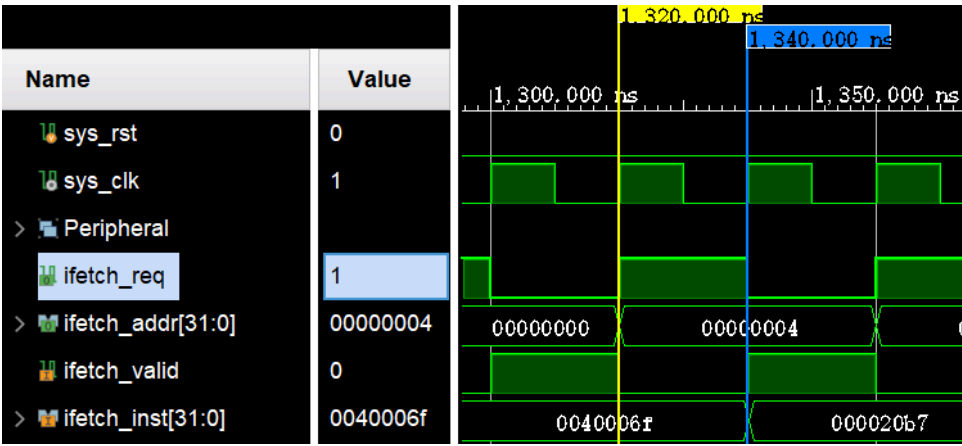


图1-5 CPU的取指波形示例

- 【1320ns】CPU需要取指时拉高 ifetch_req 信号，同时给出指令地址 ifetch_addr。ifetch_inst 信号 **仅有效一个时钟周期**。
 - 【1340ns】指令存储器接收指令地址，并在读取操作完成后，拉高 ifetch_valid 信号，告知CPU指令已经返回，同时把指令机器码通过 ifetch_inst 信号返回给CPU。
- 由lw.dump第11行可知，地址为 0x4 的指令机器码是 0x000020b7（lui x1, 0x2），与波形相符。

表1-2 CPU对外的数据访存接口信号

信号	位宽	属性	功能描述
----	----	----	------

daccess_ren	4	输出	读使能信号（有效：4'hF，无效：4'h0）
daccess_addr	32	输出	读/写地址
daccess_rvalid	1	输入	数据存储器返回的读数据有效信号 有效一个时钟表示返回一个读数据
daccess_rdata	32	输入	数据存储器返回的读数据
daccess_wen	4	输出	写使能信号，支持写字、半字、字节 （有效：非零值，无效：4'h0）
daccess_wdata	32	输出	写数据
daccess_wresp	1	输入	数据存储器返回的写响应信号 有效则表示写操作已完成

读数据时序解读

此处以miniRV的demo工程为例，介绍CPU读数据的时序。在仿真波形上任意选取一段波形，如图1-6所示。

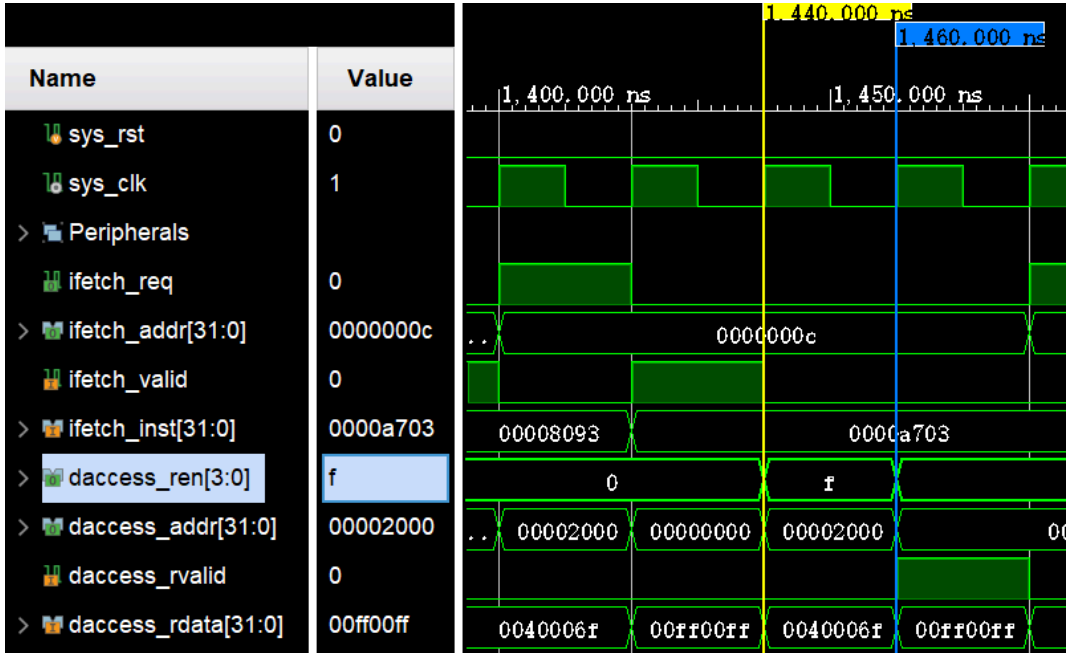


图1-6 CPU的读数据波形示例

由lw.dump第13行可知，地址为 0xc 的指令机器码是 0x0000a703（lw x14, 0(x1)），是读访存指令。

- 【1440ns】CPU需要读访存时令 daccess_ren 信号有效，同时给出读地址 daccess_addr。daccess_ren 信号 仅有效一个时钟周期。
- 【1460ns】数据存储器接收读地址，并在读取操作完成后，拉高 daccess_rvalid 信号，告知CPU数据已经返回，同时把数据通过 daccess_rdata 信号返回给CPU。

由lw.dump第231行可知，在数据段中，地址为 0x2000 的32位数据值是 0x00ff00ff，与波形相符。

3. 实例解析

下面以miniRV demo工程已实现的 addi 指令为例，结合数据通路表和控制信号表示例、仿真波形、CPU示例代码以及汇编代码，解释指令在单周期CPU中的执行过程。同学们请参照此过程自行分析和理解其他指令的执行原理。此外，还可参考实验原理-2.CPU基础架构-基本功能部件来理解demo工程CPU的各个功能部件的实现逻辑和代码。

将信号添加到仿真波形

先在左侧Scope窗口选中想要查看信号的模块，然后在旁边Objects窗口选中想要查看的信号，直接单击鼠标拖动即可将其添加到波形窗口，如图1-7所示。也可使用 ⇧ Shift 或 ⌘ Ctrl 加鼠标单击来一次选择多个信号。

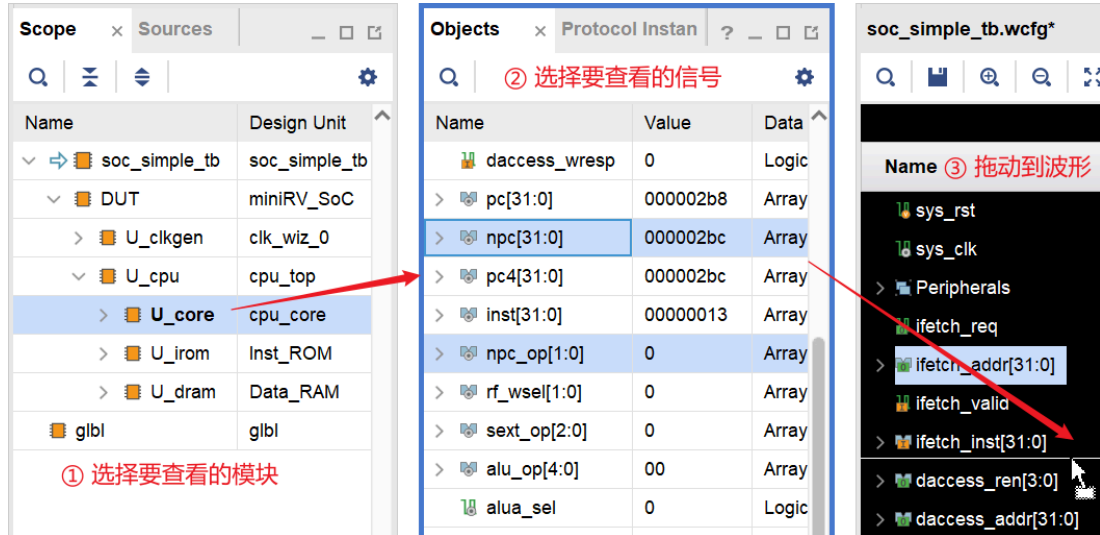


图1-7 添加信号到仿真波形窗口

在波形上定位到1840ns处，可见此时 `ifetch_req` 有效，且 `ifetch_addr` 的值为 `32'h30`，说明此时正在对地址为 `0x30` 的指令进行取指。在下一拍，即1860ns处，`ifetch_valid` 有效，且 `ifetch_inst` 的值为 `32'hf0038393`，说明地址为 `0x30` 的指令已经取到，机器码是 `0xf0038393`，如图1-8所示。

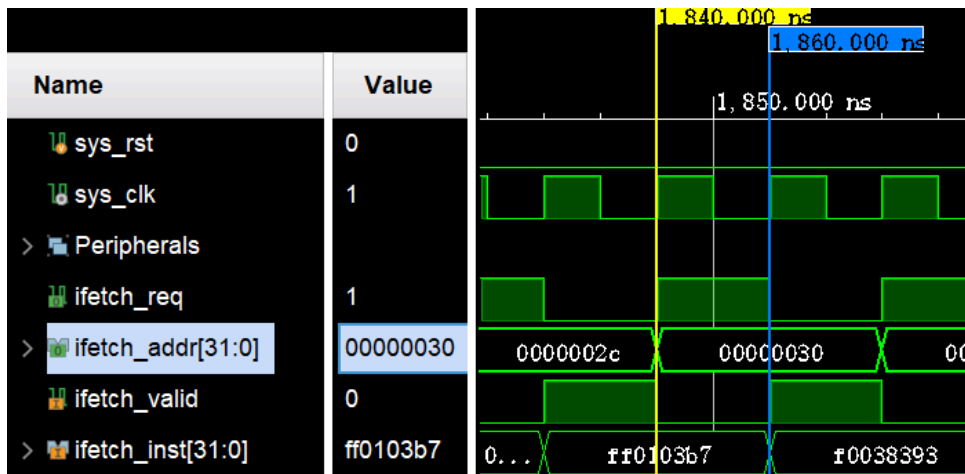


图1-8 addi 指令仿真波形示例

在 `lw.dump` 中搜索指令地址可知，地址为 `0x30` 的指令机器码是 `0xf0038393`，即 `addi x7, x7, -256` 指令，如图1-9所示，与仿真波形相符。

lw.dump 8.73 KB

23

2c: ff0103b7

lui x7,0xff010

24

30: f0038393

addi x7,x7,-256 # ff00ff00 <_end+0xff00def0>

25

34: 00300193

addi x3,x0,3

图1-9 lw.dump测试程序中的 addi 指令

取指单元

addi 指令的取指单元相关的数据通路和控制信号如图1-10所示。

所属单元	取指单元			
部件	PC	NPC		
输入信号	npc	pc	offset	br
addi	NPC.npc	PC.pc	-	-
操作选择信号	-	npc_op		
多路选择信号	-	-	-	-

指令	opcode	funct3	funct7	控制信号
				npc_op
addi	0010011	000	-	PC4

图1-10 addi 指令的数据通路和控制信号（取指单元）

由数据通路表可知，对于 addi 指令，PC的新值由NPC模块的 npc 输出信号产生，相关代码如下：

cpu_core.v

89

NPC U_NPC (

90

.op (npc_op),

91

.pc (pc),

92

.npc (npc),

93

.....

94

);

95

96

PC U_PC (

97

.....

98

.npc (npc),

99

.pc (pc)

100

);

控制器译码后产生 npc_op 控制信号，并控制NPC模块输出PC+4作为下一条指令的地址：

NPC.v

15

always @(*) begin

16

case (op)

17

`NPC_PC4: npc = pc4;

```
18         .....
19     endcase
20 end
```

由 defines.vh 头文件可知，宏定义 `NPC_PC4 的值是 2'b00：

```
defines.vh
14 `define NPC_PC4      2'b00
```

把控制器产生的 npc_op 信号、NPC产生的 npc 信号添加到波形，如图1-11所示。

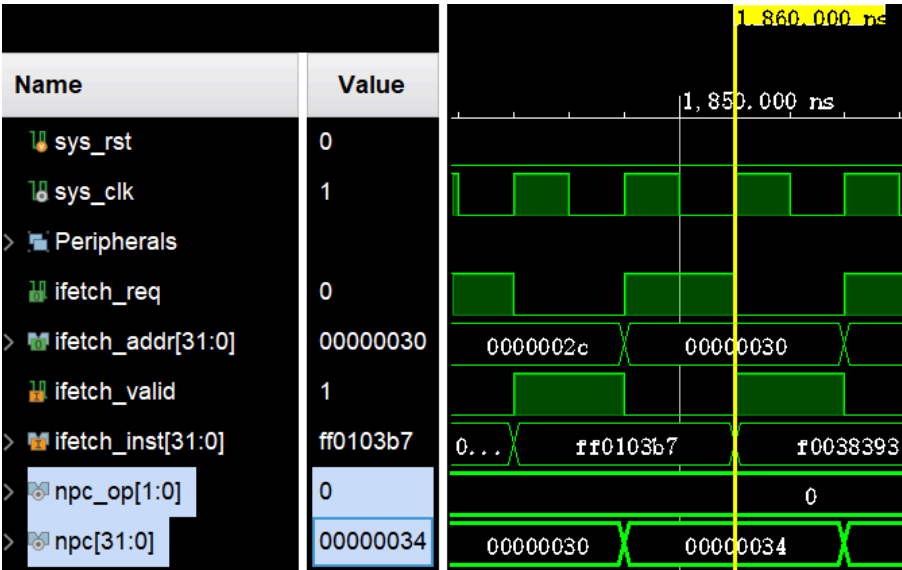


图1-11 查看 addi 指令的取指单元的相关信号

注意，只有当 ifetch_valid 有效时，ifetch_inst 才是有效的，相关信号也才是有效的。因此，我们应当观察 npc_op 和 npc 信号在1860ns的值。可见 npc_op 等于 2'b00（即 `NPC_PC4），npc 等于 32'h34（即当前PC值 0x30 再加4），与数据通路表、控制信号表及相关代码相符。

译码单元

addi 指令的译码单元相关的数据通路和控制信号如图1-12所示。

所属单元	译码单元				
部件	RF				SEXT
输入信号	rR1	rR2	wR	wD	imm
addi	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]
操作选择信号	-	-	rf_we		sext_op
多路选择信号	-	-	-	rf_wsel	-

指令	opcode	funct3	funct7	控制信号		
				rf_we	rf_wsel	sext_op
addi	0010011	000	-	1	WB_ALU	I_TYPE

图1-12 addi 指令的数据通路和控制信号（译码单元）

由数据通路表可知，addi 指令的源操作数是RS1寄存器和立即数。其中，RS1寄存器的值需要使用机器码第19位到第15位来读取RF获得，相关代码如下：

```
cpu_core.v
130   RF U_RF (
131       .....
132       .rR1      (inst[19:15]),
133       .rD1      (rf_rd1),
134       .....
135   );
```

而立即数则使用机器码第31位到第20位在SEXT中通过I类型扩展得到，相关代码如下：

```
cpu_core.v
141   SEXT U_SEXT (
142       .op      (sext_op),
143       .imm      (inst[31:7]),
144       .ext      (ext)
145   );
```

```
SEXT.v
11   always @(*) begin
12       case (op)
13       `EXT_I : ext = {{20{imm[31]}}, imm[31:20]};
14       .....
15   endcase
16   end
```

此外，由数据通路表和控制信号表可知，addi 指令需要把ALU的运算结果写回到目标寄存器，相关代码如下：

```
cpu_core.v
229   always @(*) begin
230       casex ({ld_st_flag, rf_wsel})
231       {1'b0, `WB_ALU}: rf_wD = alu_c;
232       .....
233   endcase
234   end
```

由 defines.vh 头文件可知，宏定义`EXT\_I、`WB_ALU 的值分别是 3'b000、2'b00：

```
defines.vh
18   `define EXT_I      3'b000
```

```
defines.vh
23  `define WB_ALU      2'b00
```

把控制器产生的 `sext_op`、`rf_we`、`rf_wsel` 信号，以及RS1寄存器的值（即RF的 `rD1` 输出信号）、扩展后的立即数（即SEXT的 `ext` 输出信号）添加到波形，如图1-13所示。

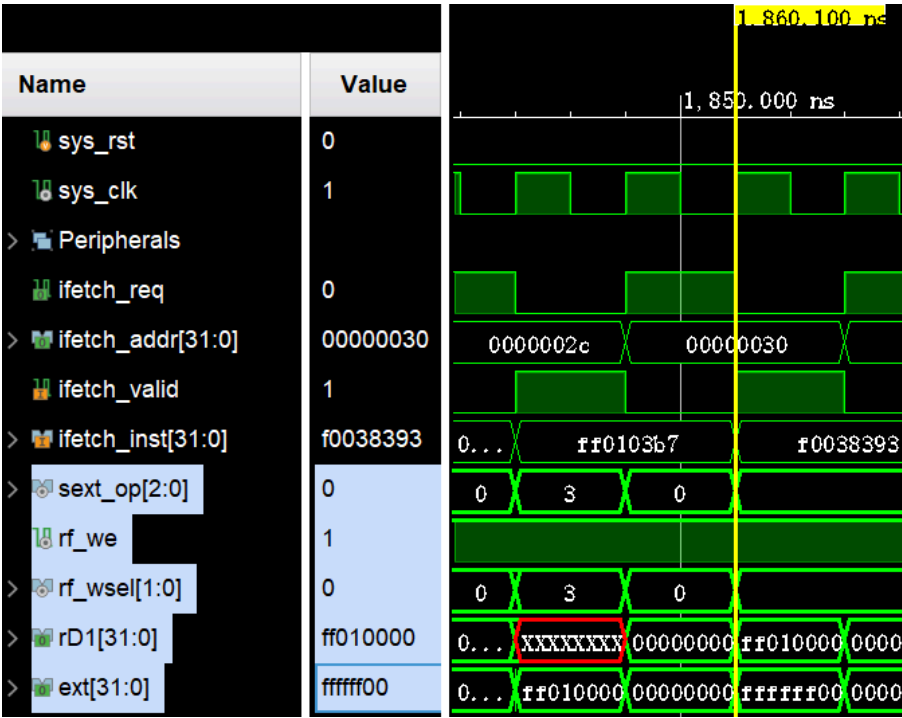


图1-13 查看 `addi` 指令的译码单元的相关信号

同样地，我们应当观察 `ifetch_valid` 信号有效时的信号值。可见 `sext_op` 等于 `3'h0`（即 `EXT_I`）、`rf_wsel` 等于0（即 `WB_ALU`）、`rf_we` 等于1表示指令要写回，与数据通路表、控制信号表及相关代码相符。

此外，由波形可知 `addi` 指令的寄存器操作数为 `0xff010000`、立即数操作数为 `0xfffff000`。

? 在 `ifetch_valid` 上升沿的瞬间，信号的值不正确？

demo工程使用Block Memory IP核作为指令存储器和数据存储器。Block Memory IP核有时会在时钟信号上升沿之后稍微迟一点才会输出指令或数据。

尽管信号发生了延迟，但指令的执行结果是在下一个时钟上升沿通过时序逻辑写回到RF的。因此，只要信号值在下一个时钟上升沿到来之前是正确的，指令的写回值就是正确的。

执行单元

`addi` 指令的执行单元相关的数据通路和控制信号如图1-14所示。

所属单元	执行单元	
部件	ALU	
输入信号	A	B
addi	RF.rD1	SEXT.ext
操作选择信号	alu_op	
多路选择信号	-	alub_sel

指令	opcode	funct3	funct7	控制信号	
				alu_op	alub_sel
addi	0010011	000	-	ADD	SEL_EXT

图1-14 addi 指令的数据通路和控制信号（执行单元）

由数据通路表可知，addi 指令需要ALU完成加法运算。加法的源数据1来自RS1寄存器，源数据2来自扩展后的立即数，相关代码如下：

cpu_core.v

```
169 assign alu_a = rf_rd1;
170 assign alu_b = alub_sel ? ext : rf_rd2;
```

ALU在控制信号 alu_op 的控制下进行加法操作：

ALU.v

```
26 always @(*) begin
27     case (op_r != 4'h0 ? op_r : op)
28         `ALU_ADD : c = a + b;
29         .....
30     endcase
31 end
```

由 defines.vh 头文件可知，宏定义 `ALU_ADD 的值是 5'h00：

defines.vh

```
8 `define ALU_ADD 5'h00
```

把控制器产生的 alu_op、alub_sel 信号，以及ALU的操作数 a、b 以及运算结果 c 添加到波形，如图1-15所示。

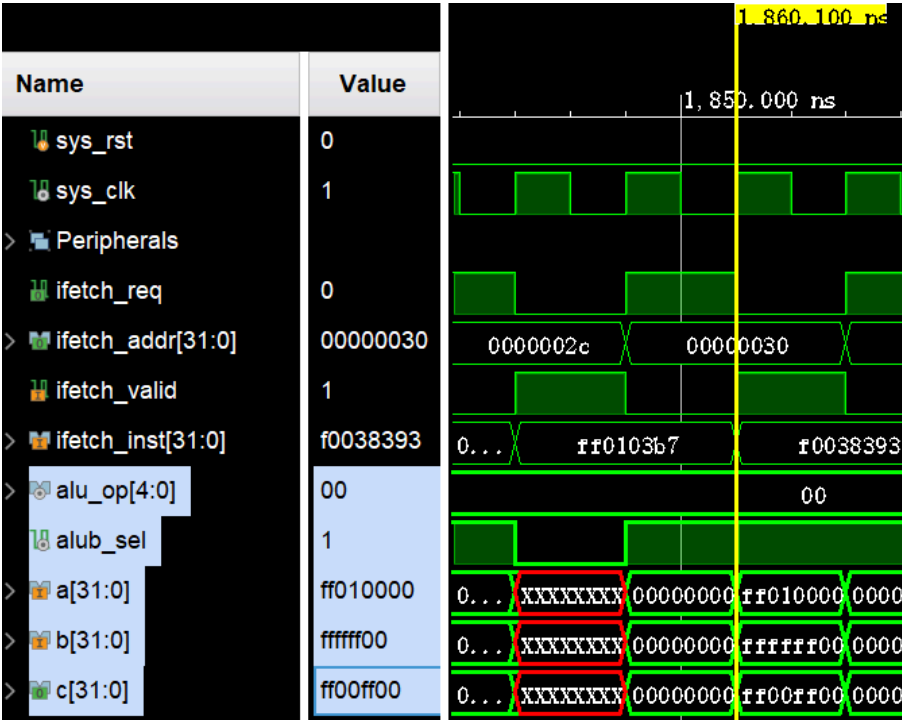


图1-15 查看 addi 指令的执行单元的相关信号

由波形可知，`alu_op` 等于 `5'h0`（即 `ALU_ADD`）、`alub_sel` 等于 1（即选择立即数作为源操作数2），与数据通路表、控制信号表及相关代码相符。此外，ALU的两个操作数与译码单元得到的两个源操作数相同。

存储单元

`addi` 指令的存储单元相关的数据通路和控制信号如图1-16所示。

所属单元	存储单元			
部件	MREQ		MEXT	
输入信号	ram_addr	ram_wdata	din	byte_offs
addi	-	-	-	-
操作选择信号	ram_rop		ram_rop	
多路选择信号	-	-	-	-

指令	opcode	funct3	funct7	控制信号 ram_rop
addi	0010011	000	-	0

图1-16 addi 指令的数据通路和控制信号（存储单元）

`addi` 指令是算术运算指令，不需要访存，因此控制信号表中的 `ram_rop` 信号为 0，对应波形如图1-17所示。

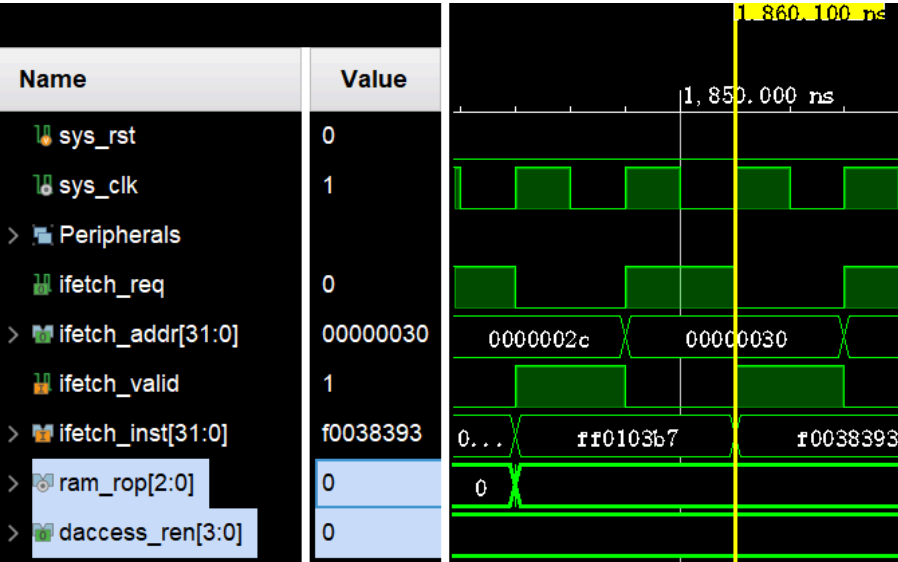


图1-17 查看 addi 指令的存储单元的相关信号

由波形可知，ram_rop 和 daccess_ren 均为0，表示指令没有发出访存请求。

miniRV指令集

本实验要求同学们基于miniRV指令集实现单周期CPU。miniRV是RV32I的子集，因而兼容RISC-V。

miniRV指令集共包含32个通用寄存器和37条指令 (含R型10条、I型15条、S型3条、B型6条、U型2条、J型1条)。

1. miniRV通用寄存器

miniRV具有32个32位通用寄存器，与RV32I完全相同，如表2-1所示。

表2-1 miniRV通用寄存器

寄存器	助记符	释义
x0	zero	常数0，永远只返回0 (硬件0，只读不可写)
x1	ra	存放调用子程序(函数)时的返回地址 (Return Address)
x2	sp	堆栈指针。对它的调整必须显式地通过指令来实现，硬件不支持堆栈指针的调整 (Stack Pointer)
x3	gp	全局指针。某些运行时系统用来为static或extern变量提供简单的访问方式 (Global Pointer)
x4	tp	线程指针。多线程应用程序通过该寄存器来访问某个线程的本地数据 (Thread Pointer)
x5~x7	t0~t2	存放临时变量，子程序(函数)使用时不自动保存这些寄存器的值，因此调用后它们的值可能被破坏
x8	s0/fp	子程序用该寄存器做堆栈帧指针。子程序(函数)必须在返回前恢复这些寄存器的值以保证其没有变化 (Saved Register / Frame Pointer)
x9	s1	子程序用寄存器。子程序(函数)必须在返回前恢复这些寄存器的值以保证其没有变化 (Saved Register)

寄存器	助记符	释义
x10~x11	a0~a1	存放子程序(函数)调用时的整型参数或返回值
x12~x17	a2~a7	存放子程序(函数)调用时的整型参数
x18~x27	s2~s11	子程序用寄存器。子程序(函数)必须在返回前恢复这些寄存器的值以保证其没有变化 (Saved Register)
x28~x31	t3~t6	存放临时变量，子程序(函数)使用时不自动保存这些寄存器的值，因此调用后它们的值可能被破坏

在汇编语言中，既可以通过寄存器号也可以通过助记符来引用一个寄存器。例如，汇编中的“x0”和“zero”均表示第0个寄存器；“x8”、“s0”和“fp”均表示第8个寄存器。

2. miniRV指令格式

miniRV指令集的指令格式与RISC-V完全相同，如图2-1所示。

	31	25 24	20 19	15 14	12 11	7 6	0
R 型	funct7	rs2	rs1	funct3	rd	opcode	
I 型	imm[11:0]		rs1	funct3	rd	opcode	
S 型	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	
B 型	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode	
U 型	imm[31:12]				rd	opcode	
J 型	imm[20 10:1 11 19:12]				rd	opcode	

图2-1 miniRV指令格式

3. miniRV指令速查表

miniRV指令的格式、功能、语法及解释汇总如表2-2所示。其中，浅蓝色背景的是A组指令，浅橙色的则是B组指令，而绿色字体的指令则是demo工程已经实现的指令。

表2-2 miniRV指令速查表

助记符	指令格式						使用语法	操作及解释
BIT#	31..25	24..20	19..15	14..12	11..7	6..0		(r)表示寄存器r的值; Mem[addr]表示地址为addr的存储单元的值; sext(a)表示a的符号扩展
R-类型	funct7	rs2	rs1	funct3	rd	opcode		
add	0000000	rs2	rs1	000	rd	0110011	add rd, rs1, rs2	(rd) ← (rs1) + (rs2)
sub	0100000	rs2	rs1	000	rd	0110011	sub rd, rs1, rs2	(rd) ← (rs1) - (rs2)
xor	0000000	rs2	rs1	100	rd	0110011	xor rd, rs1, rs2	(rd) ← (rs1) ^ (rs2)
sll	0000000	rs2	rs1	001	rd	0110011	sll rd, rs1, rs2	(rd) ← (rs1) << (rs2)
srl	0000000	rs2	rs1	101	rd	0110011	srl rd, rs1, rs2	(rd) ← (rs1) >> (rs2), 逻辑右移
sra	0100000	rs2	rs1	101	rd	0110011	sra rd, rs1, rs2	(rd) ← (rs1) >> (rs2), 算术右移
and	0000000	rs2	rs1	111	rd	0110011	and rd, rs1, rs2	(rd) ← (rs1) & (rs2)
or	0000000	rs2	rs1	110	rd	0110011	or rd, rs1, rs2	(rd) ← (rs1) (rs2)
slt	0000000	rs2	rs1	010	rd	0110011	slt rd, rs1, rs2	(rd) ← ((rs1) < (rs2)), 有符号比较
sltu	0000000	rs2	rs1	011	rd	0110011	sltu rd, rs1, rs2	(rd) ← ((rs1) < (rs2)), 无符号比较
mul	0000001	rs2	rs1	000	rd	0110011	mul rd, rs1, rs2	(rd) ← ((rs1) × (rs2))[31:0]
mulh	0000001	rs2	rs1	001	rd	0110011	mulh rd, rs1, rs2	(rd) ← ((rs1) × (rs2))[63:32], 有符号乘法
mulhu	0000001	rs2	rs1	011	rd	0110011	mulhu rd, rs1, rs2	(rd) ← ((rs1) × (rs2))[63:32], 无符号乘法
div	0000001	rs2	rs1	100	rd	0110011	div rd, rs1, rs2	(rd) ← (rs1) / (rs2), 有符号除法
divu	0000001	rs2	rs1	101	rd	0110011	divu rd, rs1, rs2	(rd) ← (rs1) / (rs2), 无符号除法
rem	0000001	rs2	rs1	110	rd	0110011	rem rd, rs1, rs2	(rd) ← (rs1) % (rs2), 有符号除法
remu	0000001	rs2	rs1	111	rd	0110011	remu rd, rs1, rs2	(rd) ← (rs1) % (rs2), 无符号除法
I-类型	imm[11:0]		rs1	funct3	rd	opcode		
addi	imm[11:0]		rs1	000	rd	0010011	addi rd, rs1, imm	(rd) ← (rs1) + sext(imm)
ori	imm[11:0]		rs1	110	rd	0010011	ori rd, rs1, imm	(rd) ← (rs1) sext(imm)
slli	0000000	shamt	rs1	001	rd	0010011	slli rd, rs1, shamt	(rd) ← (rs1) << shamt
lw	imm[11:0]		rs1	010	rd	0000011	lw rd, offset(rs1)	(rd) ← sext(Mem[(rs1) + sext(offset)] [31:0])
xori	imm[11:0]		rs1	100	rd	0010011	xori rd, rs1, imm	(rd) ← (rs1) ^ sext(imm)
srlr	0000000	shamt	rs1	101	rd	0010011	srlr rd, rs1, shamt	(rd) ← (rs1) >> shamt, 逻辑右移
srai	0100000	shamt	rs1	101	rd	0010011	srai rd, rs1, shamt	(rd) ← (rs1) >> shamt, 算术右移
lb	imm[11:0]		rs1	000	rd	0000011	lb rd, offset(rs1)	(rd) ← sext(Mem[(rs1) + sext(offset)] [7:0])
lbu	imm[11:0]		rs1	100	rd	0000011	lbu rd, offset(rs1)	(rd) ← Mem[(rs1) + sext(offset)] [7:0]
lh	imm[11:0]		rs1	001	rd	0000011	lh rd, offset(rs1)	(rd) ← sext(Mem[(rs1) + sext(offset)] [15:0])
lhu	imm[11:0]		rs1	101	rd	0000011	lhu rd, offset(rs2)	(rd) ← Mem[(rs1) + sext(offset)] [15:0]
jalr	imm[11:0]		rs1	000	rd	1100111	jalr rd, offset(rs1)	t ← (pc) + 4; (pc) ← ((rs1) + sext(offset)) & ~1; (rd) ← t
andi	imm[11:0]		rs1	111	rd	0010011	andi rd, rs1, imm	(rd) ← (rs1) & sext(imm)
slti	imm[11:0]		rs1	010	rd	0010011	slti rd, rs1, imm	(rd) ← ((rs1) < sext(imm)), 有符号比较
sltiu	imm[11:0]		rs1	011	rd	0010011	sltiu rd, rs1, imm	(rd) ← ((rs1) < sext(imm)), 无符号比较
S-类型	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode		
sb	imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb rs2, offset(rs1)	Mem[(rs1) + sext(offset)] ← (rs2) [7:0]
sh	imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	sh rs2, offset(rs1)	Mem[(rs1) + sext(offset)] ← (rs2) [15:0]
sw	imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw rs2, offset(rs1)	Mem[(rs1) + sext(offset)] ← (rs2) [31:0]
B-类型	imm[12:10:5]	rs2	rs1	funct3	imm[4:1:11]	opcode		
beq	imm[12:10:5]	rs2	rs1	000	imm[4:1:11]	1100011	beq rs1, rs2, offset	if ((rs1) = (rs2)) (pc) ← (pc) + sext(offset)
bne	imm[12:10:5]	rs2	rs1	001	imm[4:1:11]	1100011	bne rs1, rs2, offset	if ((rs1) ≠ (rs2)) (pc) ← (pc) + sext(offset)
blt	imm[12:10:5]	rs2	rs1	100	imm[4:1:11]	1100011	blt rs1, rs2, offset	if ((rs1) < (rs2)) (pc) ← (pc) + sext(offset), 有符号比较
bltu	imm[12:10:5]	rs2	rs1	110	imm[4:1:11]	1100011	bltu rs1, rs2, offset	if ((rs1) < (rs2)) (pc) ← (pc) + sext(offset), 无符号比较
bge	imm[12:10:5]	rs2	rs1	101	imm[4:1:11]	1100011	bge rs1, rs2, offset	if ((rs1) ≥ (rs2)) (pc) ← (pc) + sext(offset), 有符号比较
bgeu	imm[12:10:5]	rs2	rs1	111	imm[4:1:11]	1100011	bgeu rs1, rs2, offset	if ((rs1) ≥ (rs2)) (pc) ← (pc) + sext(offset), 无符号比较
U-类型	imm[31:12]				rd	opcode		
lui	imm[31:12]				rd	0110111	lui rd, imm	(rd) ← sext(imm[31:12] << 12)
auipc	imm[31:12]				rd	0010111	auipc rd, imm	(rd) ← (PC) + sext(imm[31:12] << 12)
J-类型	imm[20:10:11:19:12]				rd	opcode		
jal	imm[20:10:11:19:12]				rd	1101111	jal rd, offset	(rd) ← (pc) + 4; (pc) ← (pc) + sext(offset)

下面对每条指令的格式、功能等给出详细解释，所使用的符号约定如表2-3所示。

表2-3 符号释义

符号	释义
rs1、rs2、rd	32位通用寄存器的编号 (rs1、rs2存放源操作数，rd存放目的操作数)
shamt	5位移位位数
imm	12位立即数

符号	释义
offset	12位偏移量
PC	赋值符号右侧的PC表示当前指令的地址
(rs1)	寄存器rs1的值 (其他同理)
Mem[addr]	地址为addr的存储单元的值
sext(a)	a的符号扩展

4. 指令详解

4.1 R型指令

add

- 指令名：加法
- 使用语法： `add rd, rs1, rs2`
- 功能描述：把寄存器 `rs1` 加上寄存器 `rs2`，结果写入寄存器 `rd`。忽略算术溢出。
- 汇编例子：

```
add x4, x2, x3    # (x4) ← (x2) + (x3), (PC) ← (PC) + 4
```

sub

- 指令名：减法
- 使用语法： `sub rd, rs1, rs2`
- 功能描述：从寄存器 `rs1` 减去寄存器 `rs2`，结果写入寄存器 `rd`。忽略算术溢出。
- 汇编例子：

```
sub x7, x5, x9    # (x7) ← (x5) - (x9), (PC) ← (PC) + 4
```

and

- 指令名：逻辑与
- 使用语法： `and rd, rs1, rs2`

- 功能描述：将寄存器 `rs1` 和寄存器 `rs2` 进行按位与，结果写入寄存器 `rd`。
- 汇编例子：

```
and x6, x1, x0    # (x6) ← (x1) & (x0), (PC) ← (PC) + 4
```

or

- 指令名：逻辑或
- 使用语法： `or rd, rs1, rs2`
- 功能描述：将寄存器 `rs1` 和寄存器 `rs2` 进行按位或，结果写入寄存器 `rd`。
- 汇编例子：

```
or x12, x13, x1    # (x12) ← (x13) | (x1), (PC) ← (PC) + 4
```

xor

- 指令名：逻辑异或
- 使用语法： `xor rd, rs1, rs2`
- 功能描述：将寄存器 `rs1` 和寄存器 `rs2` 进行按位异或，结果写入寄存器 `rd`。
- 汇编例子：

```
xor t0, t0, t0    # (t0) ← (t0) ^ (t0), (PC) ← (PC) + 4
```

sll

- 指令名: 逻辑左移
- 使用语法： `sll rd, rs1, rs2`
- 功能描述：把寄存器 `rs1` 左移 `rs2` 位，低位补0，结果写入寄存器 `rd`。 `rs2[4:0]` 表示左移位数，高位忽略。
- 汇编例子：

```
sll t1, t6, t2    # (t1) ← (t6) << (t2), (PC) ← (PC) + 4
```

srl

- 指令名: 逻辑右移
- 使用语法： `srl rd, rs1, rs2`

- 功能描述：把寄存器 `rs1` 右移 `rs2` 位，高位补0，结果写入寄存器 `rd`。`rs2[4:0]` 表示右移位数，高位忽略。
- 汇编例子：

```
srl t5, t3, t4    # (t5) ← (t3) >> (t4), (PC) ← (PC) + 4
```

sra

- 指令名: 算术右移
- 使用语法: `sra rd, rs1, rs2`
- 功能描述：把寄存器 `rs1` 右移 `rs2` 位，高位补 `rs1[31]`，结果写入寄存器 `rd`。`rs2[4:0]` 表示右移位数，高位忽略。
- 汇编例子：

```
sra t3, t4, t7    # (t3) ← (t4) >> (t7), (PC) ← (PC) + 4
```

slt

- 指令名: 小于则置位
- 使用语法: `slt rd, rs1, rs2`
- 功能描述：比较寄存器 `rs1` 和寄存器 `rs2`，比较时视为有符号数。如果 `rs1` 更小，向寄存器 `rd` 写入1，否则写入0。
- 汇编例子：

```
slt t1, t2, t0    # (t1) ← ((t2) < (t0)), (PC) ← (PC) + 4
```

sltu

- 指令名: 无符号小于则置位
- 使用语法: `sltu rd, rs1, rs2`
- 功能描述：比较寄存器 `rs1` 和寄存器 `rs2`，比较时视为无符号数。如果 `rs1` 更小，向寄存器 `rd` 写入1，否则写入0。
- 汇编例子：

```
sltu s5, t1, s1    # (s5) ← ((t1) < (s1)), (PC) ← (PC) + 4
```

mul

- 指令名: 乘法取低32位

- 使用语法: `mul rd, rs1, rs2`
- 功能描述: 把寄存器 `rs1` 和寄存器 `rs2` 相乘, 乘积的低32位写入 `rd`。
- 汇编例子:

```
mul s5, t1, s1    # (s5) ← ((t1) × (s1))[31:0], (PC) ← (PC) + 4
```

mulh

- 指令名: 有符号乘法取高32位
- 使用语法: `mulh rd, rs1, rs2`
- 功能描述: 把寄存器 `rs1` 和寄存器 `rs2` 作有符号乘法, 乘积的高32位写入 `rd`。
- 汇编例子:

```
mulh s5, t1, s1    # (s5) ← ((t1) × (s1))[63:32], (PC) ← (PC) + 4
```

mulhu

- 指令名: 无符号乘法取高32位
- 使用语法: `mulhu rd, rs1, rs2`
- 功能描述: 把寄存器 `rs1` 和寄存器 `rs2` 作无符号乘法, 乘积的高32位写入 `rd`。
- 汇编例子:

```
mulhu s5, t1, s1    # (s5) ← ((t1) × (s1))[63:32], (PC) ← (PC) + 4
```

div

- 指令名: 有符号除法
- 使用语法: `div rd, rs1, rs2`
- 功能描述: 把寄存器 `rs1` 和寄存器 `rs2` 视为有符号数, 并计算 `rs1 ÷ rs2`, 商写入 `rd`。
- 汇编例子:

```
div s5, t1, s1    # (s5) ← (t1) / (s1), (PC) ← (PC) + 4
```

divu

- 指令名: 无符号除法
- 使用语法: `divu rd, rs1, rs2`
- 功能描述: 把寄存器 `rs1` 和寄存器 `rs2` 视为无符号数, 并计算 `rs1 ÷ rs2`, 商写入 `rd`。

- 汇编例子:

```
divu s5, t1, s1    # (s5) ← (t1) / (s1), (PC) ← (PC) + 4
```

rem

- 指令名: 有符号求余
- 使用语法: `rem rd, rs1, rs2`
- 功能描述: 把寄存器 `rs1` 和寄存器 `rs2` 视为有符号数, 并计算 `rs1 ÷ rs2`, 余数写入 `rd`。
- 汇编例子:

```
rem s5, t1, s1     # (s5) ← (t1) % (s1), (PC) ← (PC) + 4
```

remu

- 指令名: 无符号求余
- 使用语法: `remu rd, rs1, rs2`
- 功能描述: 把寄存器 `rs1` 和寄存器 `rs2` 视为无符号数, 并计算 `rs1 ÷ rs2`, 余数写入 `rd`。
- 汇编例子:

```
remu s5, t1, s1    # (s5) ← (t1) % (s1), (PC) ← (PC) + 4
```

4.2 I型指令

addi

- 指令名: 立即数加法
- 使用语法: `addi rd, rs1, imm`
- 功能描述: 把符号扩展的立即数加到寄存器 `rs1` 上, 结果写入寄存器 `rd`。忽略算术溢出。
- 汇编例子:

```
addi sp, sp, 4     # (sp) ← (sp) + 4, (PC) ← (PC) + 4
```

andi

- 指令名: 立即数逻辑与
- 使用语法: `andi rd, rs1, imm`
- 功能描述: 把符号扩展的立即数与寄存器 `rs1` 的值进行按位与, 结果写入寄存器 `rd`。

- 汇编例子:

```
andi t1, t3, 0xFFE    # (t1) ← (t3) & 0xFFE, (PC) ← (PC) + 4
```

ori

- 指令名: 立即数逻辑或
- 使用语法: `ori rd, rs1, imm`
- 功能描述: 把符号扩展的立即数与寄存器 `rs1` 的值进行按位或, 结果写入寄存器 `rd`。
- 汇编例子:

```
ori t6, t2, 0x001     # (t6) ← (t2) | 0x001, (PC) ← (PC) + 4
```

xori

- 指令名: 立即数逻辑异或
- 使用语法: `xori rd, rs1, imm`
- 功能描述: 把符号扩展的立即数与寄存器 `rs1` 的值进行按位异或, 结果写入寄存器 `rd`。
- 汇编例子:

```
xori s4, t0, 0x002    # (s4) ← (t0) ^ 0x002, (PC) ← (PC) + 4
```

slli

- 指令名: 立即数逻辑左移
- 使用语法: `slli rd, rs1, shamt`
- 功能描述: 把寄存器 `rs1` 左移 `shamt` 位, 低位补0, 结果写入寄存器 `rd`。
- 汇编例子:

```
slli a5, t3, 3        # (a5) ← (t3) << 3, (PC) ← (PC) + 4
```

srli

- 指令名: 立即数逻辑右移
- 使用语法: `srli rd, rs1, shamt`
- 功能描述: 把寄存器 `rs1` 右移 `shamt` 位, 高位补0, 结果写入寄存器 `rd`。
- 汇编例子:

```
srli a1, t2, 1    # (a1) ← (t2) >> 1, (PC) ← (PC) + 4
```

srai

- 指令名: 立即数算术右移
- 使用语法: `srai rd, rs1, shamt`
- 功能描述: 把寄存器 `rs1` 右移 `shamt` 位, 高位补 `rs1[31]`, 结果写入寄存器 `rd`。
- 汇编例子:

```
srai t3, t1, 7    # (t3) ← (t1) >> 7, (PC) ← (PC) + 4
```

slti

- 指令名: 小于立即数则置位
- 使用语法: `slti rd, rs1, imm`
- 功能描述: 比较寄存器 `rs1` 和符号扩展的立即数, 比较时视为有符号数。如果 `rs1` 更小, 向寄存器 `rd` 写入1, 否则写入0。
- 汇编例子:

```
slti t0, t5, 0    # (t0) ← ((t5) < 0), (PC) ← (PC) + 4
```

sltiu

- 指令名: 无符号小于立即数则置位
- 使用语法: `sltiu rd, rs1, imm`
- 功能描述: 比较寄存器 `rs1` 和符号扩展的立即数, 比较时视为无符号数。如果 `rs1` 更小, 向寄存器 `rd` 写入1, 否则写入0。
- 汇编例子:

```
sltiu t3, t1, -1024    # (t3) ← ((t1) < 0xC00), (PC) ← (PC) + 4
```

lb

- 指令名: 字节加载
- 使用语法: `lb rd, offset(rs1)`
- 功能描述: 从字节地址 $(rs1) + sext(offset)$ 处读取一个字节, 经符号扩展后写入寄存器 `rd`。
- 汇编例子:

```
lb t0, -3(fp)    # (t0) ← sext(Mem[(fp) - 3][7:0]), (PC) ← (PC) + 4
```

lbu

- 指令名：无符号字节加载
- 使用语法：lbu rd, offset(rs1)
- 功能描述：从字节地址 (rs1)+sext(offset) 处读取一个字节，并写入寄存器 rd。
- 汇编例子：

```
lbu t0, -3(fp)    # (t0) ← Mem[(fp) - 3][7:0], (PC) ← (PC) + 4
```

lh

- 指令名：半字加载
- 使用语法：lh rd, offset(rs1)
- 功能描述：从半字地址 (rs1)+sext(offset) 处读取2个字节，经符号扩展后写入寄存器 rd。
- 汇编例子：

```
lh t5, 6(fp)      # (t5) ← sext(Mem[(fp) + 6][15:0]), (PC) ← (PC) + 4
```

lhu

- 指令名：无符号半字加载
- 使用语法：lhu rd, offset(rs1)
- 功能描述：从半字地址 (rs1)+sext(offset) 处读取2个字节，并写入寄存器 rd。
- 汇编例子：

```
lhu t5, 6(fp)      # (t5) ← Mem[(fp) + 6][15:0], (PC) ← (PC) + 4
```

lw

- 指令名：字加载
- 使用语法：lw rd, offset(rs1)
- 功能描述：从字地址 (rs1)+sext(offset) 处读取4个字节，并写入寄存器 rd。
- 汇编例子：

```
lw t3, 8(fp)       # (t3) ← Mem[(fp) + 8], (PC) ← (PC) + 4
```


jalr

- 指令名：跳转并寄存器链接
- 使用语法：`jalr rd, offset(rs1)`
- 功能描述：将 PC 设置为 $(rs1) + sext(offset)$ ，把计算出的地址的最低位设为0，并将原 PC+4 的值写入寄存器 rd。rd 默认为1。
- 汇编例子：

```
jalr ra, -4(t0)    # t ← (PC) + 4, (PC) ← ((t0) - 4) & ~1, (ra) ← t
```

4.3 S型指令

sb

- 指令名：存字节
- 使用语法：`sb rs2, offset(rs1)`
- 功能描述：将寄存器 rs2 的最低字节存入内存的字节地址 $(rs1) + sext(offset)$ 处。
- 汇编例子：

```
sb t1, 5(fp)      # Mem[(fp) + 3] ← (t1)[7:0], (PC) ← (PC) + 4
```

sh

- 指令名：存半字
- 使用语法：`sh rs2, offset(rs1)`
- 功能描述：将寄存器 rs2 的最低2个字节存入内存的半字地址 $(rs1) + sext(offset)$ 处。
- 汇编例子：

```
sh t3, 10(fp)     # Mem[(fp) + 10] ← (t3)[15:0], (PC) ← (PC) + 4
```

sw

- 指令名：存字
- 使用语法：`sw rs2, offset(rs1)`
- 功能描述：将寄存器 rs2 存入内存的字地址 $(rs1) + sext(offset)$ 处。
- 汇编例子：

```
sw t5, 4(fp)      # Mem[(fp) + 4] ← (t5), (PC) ← (PC) + 4
```

4.4 B型指令

beq

- 指令名：相等时分支
- 使用语法：`beq rs1, rs2, offset`
- 功能描述：若寄存器 `rs1` 和寄存器 `rs2` 的值相等，则跳转。跳转的目标地址是当前PC值加上偏移量 $\text{offset} = \text{sext}(\text{offs12} \ll 1)$ 。其中，`offs12` 是指令码中的12位立即数。
- 汇编例子：

```
beq t0, t1, -4    # PC ← ((t0) == (t1)) ? PC - 4 : PC + 4
```

bne

- 指令名：不相等时分支
- 使用语法：`bne rs1, rs2, offset`
- 功能描述：若寄存器 `rs1` 和寄存器 `rs2` 的值不相等，则跳转。跳转的目标地址是当前PC值加上偏移量 $\text{offset} = \text{sext}(\text{offs12} \ll 1)$ 。其中，`offs12` 是指令码中的12位立即数。
- 汇编例子：

```
bne t0, t1, 8     # PC ← ((t0) != (t1)) ? PC + 8 : PC + 4
```

blt

- 指令名：小于时分支
- 使用语法：`blt rs1, rs2, offset`
- 功能描述：若寄存器 `rs1` 的值小于寄存器 `rs2` 的值 (比较时视为有符号数)，则跳转。跳转的目标地址是当前PC值加上偏移量 $\text{offset} = \text{sext}(\text{offs12} \ll 1)$ 。其中，`offs12` 是指令码中的12位立即数。
- 汇编例子：

```
blt t0, t1, -4    # PC ← ((t0) < (t1)) ? PC - 4 : PC + 4
```

bltu

- 指令名：无符号小于时分支
- 使用语法：`bltu rs1, rs2, offset`
- 功能描述：若寄存器 `rs1` 的值小于寄存器 `rs2` 的值 (比较时视为无符号数)，则跳转。跳转的目标地址是当前PC值加上偏移量 $\text{offset} = \text{sext}(\text{offs12} \ll 1)$ 。其中，`offs12` 是指令码中

的12位立即数。

- 汇编例子：

```
bltu t0, t1, 8    # PC ← ((t0) < (t1)) ? PC + 8 : PC + 4
```

bge

- 指令名：大于等于时分支
- 使用语法： `bge rs1, rs2, offset`
- 功能描述：若寄存器 `rs1` 的值大于等于寄存器 `rs2` 的值 (比较时视为有符号数)，则跳转。跳转的目标地址是当前PC值加上偏移量 `offset = sext(offset << 1)`。其中，`offset` 是指令码中的12位立即数。
- 汇编例子：

```
bge t0, t1, -4    # PC ← ((t0) >= (t1)) ? PC - 4 : PC + 4
```

bgeu

- 指令名：无符号大于等于时分支
- 使用语法： `bgeu rs1, rs2, offset`
- 功能描述：若寄存器 `rs1` 的值大于等于寄存器 `rs2` 的值 (比较时视为无符号数)，则跳转。跳转的目标地址是当前PC值加上偏移量 `offset = sext(offset << 1)`。其中，`offset` 是指令码中的12位立即数。
- 汇编例子：

```
bgeu t0, t1, 8    # PC ← ((t0) >= (t1)) ? PC + 8 : PC + 4
```

4.5 U型指令

lui

- 指令名：高位立即数加载
- 使用语法： `lui rd, imm`
- 功能描述：将符号扩展的20位立即数逻辑左移12位，结果写入寄存器 `rd`。
- 汇编例子：

```
lui t0, 0xABCDE    # (t0) ← 0xABCDE000, (PC) ← (PC) + 4
```

auipc

- 指令名：PC加立即数
- 使用语法：auipc rd, imm
- 功能描述：把符号扩展的20位立即数逻辑左移12位，再与 PC 相加，结果写入寄存器 rd。
- 汇编例子：

```
auipc t3, 0x2468A    # (t3) ← (PC) + 0x2468A000, (PC) ← (PC) + 4
```

4.6 J型指令

jal

- 指令名：跳转并链接
- 使用语法：jal rd, offset
- 功能描述：将 PC+4 的值写入寄存器 rd（rd 默认为1），然后无条件跳转。跳转的目标地址是当前 PC 值加上偏移量 $offset = sext(offset_{20} \ll 1)$ 。其中，offset₂₀ 是指令码中的20位立即数。
- 汇编例子：

```
jal ra, -4    # (ra) ← (PC) + 4, (PC) ← (PC) - 4
```

miniLA指令集

芯片和处理器设计是制约我国科学技术进一步发展的“卡脖子”技术之一。尽管我国在军事、超级计算机和航空航天等关键领域已经逐步实现芯片和处理器的自主研发和设计，但与国外相比，我国在芯片和处理器设计以及软件生态建设等方面仍然存在较大短板。

为了进一步实现芯片和处理器技术的自主可控，龙芯中科基于二十多年的CPU研制和生态建设的积累，于2020年推出了LoongArch指令集架构。该架构包括基础架构和向量指令、虚拟化、二进制翻译等扩展部分，具备良好的自主性、先进性和兼容性。

miniLA是龙芯LA32R (LoongArch32 Reduced) 指令集的子集，共包含32个通用寄存器和38条指令 (含3R型10条、2RI5型3条、2RI12型14条、1RI20型2条、2RI16型7条和I26型2条)。

1. miniLA通用寄存器

miniLA具有32个32位通用寄存器，与LA32R完全相同，如表2-1所示。

表2-1 miniLA通用寄存器

寄存器	助记符	释义
r0	zero	常数0，永远只返回0 (硬件0，只读不可写)
r1	ra	存放调用子程序(函数)时的返回地址 (Return Address)
r2	tp	线程指针。多线程应用程序通过该寄存器来访问某个线程的本地数据 (Thread Pointer)
r3	sp	堆栈指针。对它的调整必须显式地通过指令来实现，硬件不支持堆栈指针的调整 (Stack Pointer)
r4~r5	a0~a1	存放子程序(函数)调用时的整型参数或返回值
r6~r11	a2~a7	存放子程序(函数)调用时的整型参数
r12~r20	t0~t8	存放临时变量，子程序(函数)使用时不自动保存这些寄存器的值，因此调用后它们的值可能被破坏

寄存器	助记符	释义
r21	u0	其他通用寄存器
r22	fp / s9	子程序用该寄存器做堆栈帧指针。子程序(函数)必须在返回前恢复这些寄存器的值以保证其没有变化 (Frame Pointer)。或作为静态寄存器
r23~r31	s0~s8	静态寄存器

在miniLA架构中，指令集与通用寄存器存在正交关系，即所有的32个通用寄存器都可以被任意指令使用。

另外，在汇编语言中，既可以通过寄存器号也可以通过助记符来引用一个寄存器。例如，汇编中的“r0”和“zero”均表示第0个寄存器；“r3”和“sp”均表示第3个寄存器。



小贴士

在LoongArch汇编程序使用寄存器时，有时需添加 \$ 符号（如 `$r0`、`$zero`），否则相关汇编工具会报语法错误。

2. miniLA指令速查表

miniLA指令的格式、功能、语法及解释汇总如表2-2A所示。其中，浅蓝色背景的是A组指令，浅橙色的则是B组指令，而绿色字体的指令则是demo工程已经实现的指令。

表2-2 miniLA指令速查表

助记符	指令格式							使用语法	操作及解释
BIT#	31..26	25	24..22	21..15	14..10	9..5	4..0		(r)表示寄存器r的值; Mem[addr]表示地址为addr的存储单元的值; sext(a)表示a的符号扩展; zext(a)表示a的零扩展
3R	opcode				rk	rj	rd		
add.w	000000	0	000	0100000	rk	rj	rd	add.w rd, rj, rk	$(rd) \leftarrow (rj) + (rk)$
sub.w	000000	0	000	0100010	rk	rj	rd	sub.w rd, rj, rk	$(rd) \leftarrow (rj) - (rk)$
xor	000000	0	000	0101011	rk	rj	rd	xor rd, rj, rk	$(rd) \leftarrow (rj) \wedge (rk)$
sll.w	000000	0	000	0101110	rk	rj	rd	sll.w rd, rj, rk	$(rd) \leftarrow (rj) \ll (rk)[4:0]$
srl.w	000000	0	000	0101111	rk	rj	rd	srl.w rd, rj, rk	$(rd) \leftarrow (rj) \gg (rk)[4:0]$, 逻辑右移
sra.w	000000	0	000	0110000	rk	rj	rd	sra.w rd, rj, rk	$(rd) \leftarrow (rj) \gg (rk)[4:0]$, 算术右移
and	000000	0	000	0101001	rk	rj	rd	and rd, rj, rk	$(rd) \leftarrow (rj) \& (rk)$
or	000000	0	000	0101010	rk	rj	rd	or rd, rj, rk	$(rd) \leftarrow (rj) (rk)$
slt	000000	0	000	0100100	rk	rj	rd	slt rd, rj, rk	$(rd) \leftarrow ((rj) < (rk))$, 有符号比较
sltu	000000	0	000	0100101	rk	rj	rd	sltu rd, rj, rk	$(rd) \leftarrow ((rj) < (rk))$, 无符号比较
mul.w	000000	0	000	0111000	rk	rj	rd	mul.w rd, rj, rk	$(rd) \leftarrow ((rj) \times (rk))[31:0]$
mulh.w	000000	0	000	0111001	rk	rj	rd	mulh.w rd, rj, rk	$(rd) \leftarrow ((rj) \times (rk))[63:32]$, 有符号乘法
mulh.wu	000000	0	000	0111010	rk	rj	rd	mulh.wu rd, rj, rk	$(rd) \leftarrow ((rj) \times (rk))[63:32]$, 无符号乘法
div.w	000000	0	000	1000000	rk	rj	rd	div.w rd, rj, rk	$(rd) \leftarrow (rj) / (rk)$, 有符号除法
div.wu	000000	0	000	1000010	rk	rj	rd	div.wu rd, rj, rk	$(rd) \leftarrow (rj) / (rk)$, 无符号除法
mod.w	000000	0	000	1000001	rk	rj	rd	mod.w rd, rj, rk	$(rd) \leftarrow (rj) \% (rk)$, 有符号除法
mod.wu	000000	0	000	1000011	rk	rj	rd	mod.wu rd, rj, rk	$(rd) \leftarrow (rj) \% (rk)$, 无符号除法
2RI5	opcode				ui5[4:0]	rj	rd		
slli.w	000000	0	001	0000001	ui5[4:0]	rj	rd	slli.w rd, rj, ui5	$(rd) \leftarrow (rj) \ll ui5$
srli.w	000000	0	001	0001001	ui5[4:0]	rj	rd	srli.w rd, rj, ui5	$(rd) \leftarrow (rj) \gg ui5$, 逻辑右移
srai.w	000000	0	001	0010001	ui5[4:0]	rj	rd	srai.w rd, rj, ui5	$(rd) \leftarrow (rj) \gg ui5$, 算术右移
2RI12	opcode				si12/ui12	rj	rd		
addi.w	000000	1	010	si12[11:0]		rj	rd	addi.w rd, rj, si12	$(rd) \leftarrow (rj) + sext(si12)$
ori	000000	1	110	ui12[11:0]		rj	rd	ori rd, rj, si12	$(rd) \leftarrow (rj) zext(ui12)$
ld.w	001010	0	010	si12[11:0]		rj	rd	ld.w rd, rj, si12	$(rd) \leftarrow Mem[(rj) + sext(si12)]$
xori	000000	1	111	ui12[11:0]		rj	rd	xori rd, rj, si12	$(rd) \leftarrow (rj) \wedge zext(ui12)$
ld.b	001010	0	000	si12[11:0]		rj	rd	ld.b rd, rj, si12	$(rd) \leftarrow sext(Mem[(rj) + sext(si12)][7:0])$
ld.bu	001010	1	000	si12[11:0]		rj	rd	ld.bu rd, rj, si12	$(rd) \leftarrow Mem[(rj) + sext(si12)][7:0]$
ld.h	001010	0	001	si12[11:0]		rj	rd	ld.h rd, rj, si12	$(rd) \leftarrow sext(Mem[(rj) + sext(si12)][15:0])$
ld.hu	001010	1	001	si12[11:0]		rj	rd	ld.hu rd, rj, si12	$(rd) \leftarrow Mem[(rj) + sext(si12)][15:0]$
st.b	001010	0	100	si12[11:0]		rj	rd	st.b rd, rj, si12	$Mem[(rj) + sext(si12)] \leftarrow (rd)[7:0]$
st.h	001010	0	101	si12[11:0]		rj	rd	st.h rd, rj, si12	$Mem[(rj) + sext(si12)] \leftarrow (rd)[15:0]$
st.w	001010	0	110	si12[11:0]		rj	rd	st.w rd, rj, si12	$Mem[(rj) + sext(si12)] \leftarrow (rd)[31:0]$
andi	000000	1	101	ui12[11:0]		rj	rd	andi rd, rj, si12	$(rd) \leftarrow (rj) \& zext(ui12)$
slti	000000	1	000	si12[11:0]		rj	rd	slti rd, rj, si12	$(rd) \leftarrow ((rj) < sext(si12))$, 有符号比较
sltui	000000	1	001	si12[11:0]		rj	rd	sltui rd, rj, si12	$(rd) \leftarrow ((rj) < sext(si12))$, 无符号比较
1RI20	opcode				si20[31:12]		rd		
lui2i.w	000101	0		si20[31:12]			rd	lui2i.w rd, si20	$(rd) \leftarrow si20[31:12] \ll 12$
pcaddu12i	000111	0		si20[31:12]			rd	pcaddu12i rd, si20	$(rd) \leftarrow (PC) + (si20[31:12] \ll 12)$
2RI16	opcode				offs[17:2]		rj	rd	
beq	010110			offs[17:2]		rj	rd	beq rj, rd, offs16	if $((rj) = (rd))$ (pc) $\leftarrow$ (pc) + sext({offs, 2'b0})
bne	010111			offs[17:2]		rj	rd	bne rj, rd, offs16	if $((rj) \neq (rd))$ (pc) $\leftarrow$ (pc) + sext({offs, 2'b0})
jirl	010011			offs[17:2]		rj	rd	jirl rd, rj, offs16	$(rd) \leftarrow (pc) + 4; (pc) \leftarrow ((rj) + sext(\{offs, 2'b0\}))$
blt	011000			offs[17:2]		rj	rd	blt rj, rd, offs16	if $((rj) < (rd))$ (pc) $\leftarrow$ (pc) + sext({offs, 2'b0}), 有符号比较
bltu	011010			offs[17:2]		rj	rd	bltu rj, rd, offs16	if $((rj) < (rd))$ (pc) $\leftarrow$ (pc) + sext({offs, 2'b0}), 无符号比较
bge	011001			offs[17:2]		rj	rd	bge rj, rd, offs16	if $((rj) \geq (rd))$ (pc) $\leftarrow$ (pc) + sext({offs, 2'b0}), 有符号比较
bgeu	011011			offs[17:2]		rj	rd	bgeu rj, rd, offs16	if $((rj) \geq (rd))$ (pc) $\leftarrow$ (pc) + sext({offs, 2'b0}), 无符号比较
l26	opcode				offs[17:2]		offs[27:18]		
b	010100			offs[17:2]			offs[27:18]	b offs26	$(pc) \leftarrow (pc) + sext(\{offs, 2'b0\})$
bl	010101			offs[17:2]			offs[27:18]	bl offs26	$(r1) \leftarrow (pc) + 4; (pc) \leftarrow (pc) + sext(\{offs, 2'b0\})$

下面对每条指令的格式、功能等给出详细解释，所使用的符号约定如表2-3所示。

表2-3 符号释义

符号	释义
rj、rk、rd	32位通用寄存器的编号 (rj、rk存放源操作数，rd存放目的操作数)
ui5	5位无符号立即数

符号	释义
si12	12位有符号立即数
si20	20位有符号立即数
offset	指令所使用的立即数 (扩展后)
offsXX	指令码中的立即数 (XX是表示位宽的数字；扩展前)
PC	赋值符号右侧的PC表示当前指令的地址
(rj)	寄存器rj的值 (其他同理)
Mem[addr]	地址为addr的存储单元的值
sext(a)	a的符号扩展
zext(a)	a的零扩展

3. 指令详解

3.1 3R型指令

add.w

- 指令名：加法
- 使用语法： `add.w rd, rj, rk`
- 功能描述：把寄存器 `rj` 加上寄存器 `rk`，结果写入寄存器 `rd`。忽略算术溢出。
- 汇编例子：

```
add.w r4, r2, r3    # (x4) ← (x2) + (x3), (PC) ← (PC) + 4
```

sub.w

- 指令名：减法
- 使用语法： `sub.w rd, rj, rk`
- 功能描述：从寄存器 `rj` 减去寄存器 `rk`，结果写入寄存器 `rd`。忽略算术溢出。

- 汇编例子：

```
sub r7, r5, r9    # (r7) ← (r5) - (r9), (PC) ← (PC) + 4
```

and

- 指令名：逻辑与
- 使用语法：and rd, rj, rk
- 功能描述：将寄存器 rj 和寄存器 rk 进行按位与，结果写入寄存器 rd。
- 汇编例子：

```
and r6, r1, r0    # (r6) ← (r1) & (r0), (PC) ← (PC) + 4
```

or

- 指令名：逻辑或
- 使用语法：or rd, rj, rk
- 功能描述：将寄存器 rj 和寄存器 rk 进行按位或，结果写入寄存器 rd。
- 汇编例子：

```
or r12, r13, r1   # (r12) ← (r13) | (r1), (PC) ← (PC) + 4
```

xor

- 指令名：逻辑异或
- 使用语法：xor rd, rj, rk
- 功能描述：将寄存器 rj 和寄存器 rk 进行按位异或，结果写入寄存器 rd。
- 汇编例子：

```
xor r3, r3, r3    # (r3) ← (r3) ^ (r3), (PC) ← (PC) + 4
```

sll.w

- 指令名：逻辑左移
- 使用语法：sll.w rd, rj, rk
- 功能描述：把寄存器 rj 左移 rk 位，低位补0，结果写入寄存器 rd。rk[4:0] 表示左移位数，高位忽略。
- 汇编例子：

```
sll.w r1, r6, r2    # (r1) ← (r6) << (r2)[4:0], (PC) ← (PC) + 4
```

srl.w

- 指令名: 逻辑右移
- 使用语法: `srl.w rd, rj, rk`
- 功能描述: 把寄存器 `rj` 右移 `rk` 位, 高位补0, 结果写入寄存器 `rd`。 `rk[4:0]` 表示右移位数, 高位忽略。
- 汇编例子:

```
srl.w r5, r3, r4    # (r5) ← (r3) >> (r4)[4:0], (PC) ← (PC) + 4
```

sra.w

- 指令名: 算术右移
- 使用语法: `sra.w rd, rj, rk`
- 功能描述: 把寄存器 `rj` 右移 `rk` 位, 高位补 `rj[31]`, 结果写入寄存器 `rd`。 `rk[4:0]` 表示右移位数, 高位忽略。
- 汇编例子:

```
sra.w r3, r4, r7    # (r3) ← (r4) >> (r7)[4:0], (PC) ← (PC) + 4
```

slt

- 指令名: 小于则置位
- 使用语法: `slt rd, rj, rk`
- 功能描述: 比较寄存器 `rj` 和寄存器 `rk`, 比较时视为有符号数。如果 `rj` 更小, 向寄存器 `rd` 写入1, 否则写入0。
- 汇编例子:

```
slt r1, r2, r0      # (r1) ← ((r2) < (r0)), (PC) ← (PC) + 4
```

sltu

- 指令名: 无符号小于则置位
- 使用语法: `sltu rd, rj, rk`
- 功能描述: 比较寄存器 `rj` 和寄存器 `rk`, 比较时视为无符号数。如果 `rj` 更小, 向寄存器 `rd` 写入1, 否则写入0。

- 汇编例子：

```
sltu r5, r1, r2    # (r5) ← ((r1) < (r2)), (PC) ← (PC) + 4
```

mul.w

- 指令名: 乘法取低32位
- 使用语法: `mul.w rd, rj, rk`
- 功能描述: 把寄存器 `rj` 和寄存器 `rk` 相乘, 乘积的低32位写入 `rd`。
- 汇编例子:

```
mul.w r5, r1, r2    # (r5) ← ((r1) × (r2))[31:0], (PC) ← (PC) + 4
```

mulh.w

- 指令名: 有符号乘法取高32位
- 使用语法: `mulh.w rd, rj, rk`
- 功能描述: 把寄存器 `rj` 和寄存器 `rk` 作有符号乘法, 乘积的高32位写入 `rd`。
- 汇编例子:

```
mulh.w r5, r1, r2    # (r5) ← ((r1) × (r2))[63:32], (PC) ← (PC) + 4
```

mulh.wu

- 指令名: 无符号乘法取高32位
- 使用语法: `mulh.wu rd, rj, rk`
- 功能描述: 把寄存器 `rj` 和寄存器 `rk` 作无符号乘法, 乘积的高32位写入 `rd`。
- 汇编例子:

```
mulh.wu r5, r1, r2    # (r5) ← ((r1) × (r2))[63:32], (PC) ← (PC) + 4
```

div.w

- 指令名: 有符号除法
- 使用语法: `div.w rd, rj, rk`
- 功能描述: 把寄存器 `rj` 和寄存器 `rk` 视为有符号数, 并计算 $rj \div rk$, 商写入 `rd`。
- 汇编例子:

```
div.w r5, r1, r2    # (r5) ← (r1) / (r2), (PC) ← (PC) + 4
```

div.wu

- 指令名: 无符号除法
- 使用语法: `div.wu rd, rj, rk`
- 功能描述: 把寄存器 `rj` 和寄存器 `rk` 视为无符号数, 并计算 $rj \div rk$, 商写入 `rd`。
- 汇编例子:

```
div.wu r5, r1, r2    # (r5) ← (r1) / (r2), (PC) ← (PC) + 4
```

mod.w

- 指令名: 有符号求余
- 使用语法: `mod.w rd, rj, rk`
- 功能描述: 把寄存器 `rj` 和寄存器 `rk` 视为有符号数, 并计算 $rj \div rk$, 余数写入 `rd`。
- 汇编例子:

```
mod.w r5, r1, r2    # (r5) ← (r1) % (r2), (PC) ← (PC) + 4
```

mod.wu

- 指令名: 无符号求余
- 使用语法: `mod.wu rd, rj, rk`
- 功能描述: 把寄存器 `rj` 和寄存器 `rk` 视为无符号数, 并计算 $rj \div rk$, 余数写入 `rd`。
- 汇编例子:

```
mod.wu r5, r1, r2    # (r5) ← (r1) % (r2), (PC) ← (PC) + 4
```

3.2 2RI5型指令

slli.w

- 指令名: 立即数逻辑左移
- 使用语法: `slli.w rd, rj, ui5`
- 功能描述: 把寄存器 `rj` 左移 `ui5` 位, 低位补0, 结果写入寄存器 `rd`。
- 汇编例子:

```
slli.w r5, r3, 3    # (r5) ← (r3) << 3, (PC) ← (PC) + 4
```

srli.w

- 指令名：立即数逻辑右移
- 使用语法：srli.w rd, rj, ui5
- 功能描述：把寄存器 rj 右移 ui5 位，高位补0，结果写入寄存器 rd。
- 汇编例子：

```
srli.w r1, r2, 1    # (r1) ← (r2) >> 1, (PC) ← (PC) + 4
```

srai.w

- 指令名：立即数算术右移
- 使用语法：srai.w rd, rj, ui5
- 功能描述：把寄存器 rj 右移 ui5 位，高位补 rj[31]，结果写入寄存器 rd。
- 汇编例子：

```
srai.w r3, r1, 7    # (r3) ← (r1) >> 7, (PC) ← (PC) + 4
```

3.3 2RI12型指令

addi.w

- 指令名：立即数加法
- 使用语法：addi.w rd, rj, si12
- 功能描述：把符号扩展的立即数加到寄存器 rj 上，结果写入寄存器 rd。忽略算术溢出。
- 汇编例子：

```
addi.w r3, r3, 4    # (r3) ← (r3) + 4, (PC) ← (PC) + 4
```

andi

- 指令名：立即数逻辑与
- 使用语法：andi rd, rj, ui12
- 功能描述：把零扩展的立即数与寄存器 rj 进行按位与，结果写入寄存器 rd。
- 汇编例子：

```
andi r1, r3, 0xFFE    # (r1) ← (r3) & 0x0000_0FFE, (PC) ← (PC) + 4
```

ori

- 指令名：立即数逻辑或
- 使用语法：ori rd, rj, ui12
- 功能描述：把零扩展的立即数与寄存器 rj 的值进行按位或，结果写入寄存器 rd。
- 汇编例子：

```
ori r6, r2, 0x001    # (r6) ← (r2) | 0x001, (PC) ← (PC) + 4
```

xori

- 指令名：立即数逻辑异或
- 使用语法：xori rd, rj, ui12
- 功能描述：把零扩展的立即数与寄存器 rj 的值进行按位异或，结果写入寄存器 rd。
- 汇编例子：

```
xori r4, r0, 0x002    # (r4) ← (r0) ^ 0x002, (PC) ← (PC) + 4
```

slti

- 指令名：小于立即数则置位
- 使用语法：slti rd, rj, si12
- 功能描述：比较寄存器 rj 和符号扩展的立即数，比较时视为有符号数。如果 rj 更小，向寄存器 rd 写入1，否则写入0。
- 汇编例子：

```
slti r7, r5, 0        # (r7) ← ((r5) < 0), (PC) ← (PC) + 4
```

sltui

- 指令名：无符号小于立即数则置位
- 使用语法：sltui rd, rj, si12
- 功能描述：比较寄存器 rj 和符号扩展的立即数，比较时视为无符号数。如果 rj 更小，向寄存器 rd 写入1，否则写入0。
- 汇编例子：

```
sltui r3, r1, -1024    # (r3) ← ((r1) < 0xC00), (PC) ← (PC) + 4
```

ld.b

- 指令名：字节加载
- 使用语法：ld.b rd, rj, si12
- 功能描述：从字节地址 (rj)+sext(si12) 处读取一个字节，经符号扩展后写入寄存器 rd。
- 汇编例子：

```
ld.b r5, r2, -3    # (r5) ← sext(Mem[(r2) - 3][7:0]), (PC) ← (PC) + 4
```

ld.bu

- 指令名：无符号字节加载
- 使用语法：ld.bu rd, rj, si12
- 功能描述：从字节地址 (rj)+sext(si12) 处读取一个字节，并写入寄存器。
- 汇编例子：

```
ld.bu r5, r2, -3    # (r5) ← Mem[(r2) - 3][7:0], (PC) ← (PC) + 4
```

ld.h

- 指令名：半字加载
- 使用语法：ld.h rd, rj, si12
- 功能描述：从半字地址 (rj)+sext(si12) 处读取2个字节，经符号扩展后写入寄存器。
- 汇编例子：

```
ld.h r5, r2, 1    # (r5) ← sext(Mem[(r2) + 1][15:0]), (PC) ← (PC) + 4
```

ld.hu

- 指令名：无符号半字加载
- 使用语法：ld.hu rd, rj, si12
- 功能描述：从半字地址 (rj)+sext(si12) 处读取2个字节，并写入寄存器 rd。
- 汇编例子：

```
ld.hu r5, r2, 6    # (r5) ← Mem[(r2) + 6][15:0], (PC) ← (PC) + 4
```

ld.w

- 指令名：字加载
- 使用语法：ld.w rd, rj, si12
- 功能描述：从字地址 (rj)+sext(si12) 处读取4个字节，并写入寄存器 rd。
- 汇编例子：

```
ld.w r5, r2, 8    # (r5) ← Mem[(r2) + 8], (PC) ← (PC) + 4
```

st.b

- 指令名：存字节
- 使用语法：st.b rd, rj, si12
- 功能描述：将寄存器 rd 的最低字节存入内存的字节地址 (rj)+sext(si12) 处。
- 汇编例子：

```
st.b r9, r11, 5    # Mem[(r11) + 5] ← (r9)[7:0], (PC) ← (PC) + 4
```

st.h

- 指令名：存半字
- 使用语法：st.h rd, rj, si12
- 功能描述：将寄存器 rd 的最低2个字节存入内存的半字地址 (rj)+sext(si12) 处。
- 汇编例子：

```
st.h r9, r11, 10    # Mem[(r11) + 10] ← (r9)[15:0], (PC) ← (PC) + 4
```

st.w

- 指令名：存字
- 使用语法：st.w rd, rj, si12
- 功能描述：将寄存器 rd 存入内存的字地址 (rj)+sext(si12) 处。
- 汇编例子：

```
st.w r9, r11, 4    # Mem[(r11) + 4] ← (r9), (PC) ← (PC) + 4
```

3.4 1RI20型指令

lu12i.w

- 指令名：高位立即数加载
- 使用语法：lu12i.w rd, si20
- 功能描述：将符号扩展的20位立即数逻辑左移12位，结果写入寄存器 rd。
- 汇编例子：

```
lu12i.w r15, 0xABCDE    # (r15) ← 0xABCDE000, (PC) ← (PC) + 4
```

pcaddu12i

- 指令名：PC加立即数
- 使用语法：pcaddu12i rd, si20
- 功能描述：把符号扩展的20位立即数逻辑左移12位，再与 PC 相加，结果写入寄存器 rd。
- 汇编例子：

```
pcaddu12i r6, 0x2468A    # (r6) ← (PC) + 0x2468A000, (PC) ← (PC) + 4
```

3.5 2RI16型指令

beq

- 指令名：相等时分支
- 使用语法：beq rj, rd, offset
- 功能描述：若寄存器 rj 和寄存器 rd 的值相等，则跳转。跳转的目标地址是当前 PC 值加上偏移量 $\text{offset} = \text{sext}(\text{offs16} \ll 2)$ 。其中，offs16 是指令码中的16位立即数。
- 汇编例子：

```
beq r0, r1, -4    # PC ← ((r0) == (r1)) ? PC - 4 : PC + 4
```

bne

- 指令名：不相等时分支
- 使用语法：bne rj, rd, offset
- 功能描述：若寄存器 rj 和寄存器 rd 的值不相等，则跳转。跳转的目标地址是当前 PC 值加上偏移量 $\text{offset} = \text{sext}(\text{offs16} \ll 2)$ 。其中，offs16 是指令码中的16位立即数。
- 汇编例子：

```
bne r0, r1, 8    # PC ← ((r0) != (r1)) ? PC + 8 : PC + 4
```

blt

- 指令名：小于时分支
- 使用语法： `blt rj, rd, offset`
- 功能描述：若寄存器 `rj` 的值小于寄存器 `rd` 的值（比较时视为有符号数），则跳转。跳转的目标地址是当前 PC 值加上偏移量 `offset = sext(offset16 << 2)`。其中，`offset16` 是指令码中的16位立即数。
- 汇编例子：

```
blt r7, r0, -4    # PC ← ((r7) < (r0)) ? PC - 4 : PC + 4
```

bltu

- 指令名：无符号小于时分支
- 使用语法： `bltu rj, rd, offset`
- 功能描述：若寄存器 `rj` 的值小于寄存器 `rd` 的值（比较时视为无符号数），则跳转。跳转的目标地址是当前 PC 值加上偏移量 `offset = sext(offset16 << 2)`。其中，`offset16` 是指令码中的16位立即数。
- 汇编例子：

```
bltu r7, r0, 8    # PC ← ((r7) < (r0)) ? PC + 8 : PC + 4
```

bge

- 指令名：大于等于时分支
- 使用语法： `bge rj, rd, offset`
- 功能描述：若寄存器 `rj` 的值大于等于寄存器 `rd` 的值（比较时视为有符号数），则跳转。跳转的目标地址是当前 PC 值加上偏移量 `offset = sext(offset16 << 2)`。其中，`offset16` 是指令码中的16位立即数。
- 汇编例子：

```
bge r6, r0, -4    # PC ← ((r6) >= (r0)) ? PC - 4 : PC + 4
```

bgeu

- 指令名：无符号大于等于时分支
- 使用语法： `bgeu rj, rd, offset`

- 功能描述：若寄存器 r_j 的值大于等于寄存器 r_d 的值（比较时视为无符号数），则跳转。跳转的目标地址是当前 PC 值加上偏移量 $offset = sext(offsets16 \ll 2)$ 。其中， $offsets16$ 是指令码中的16位立即数。
- 汇编例子：

```
bgeu r10, r0, 8    # PC ← ((r10) >= (r0)) ? PC + 8 : PC + 4
```

jirl

- 指令名：无条件跳转并寄存器链接
- 使用语法：jirl $r_d, r_j, offset$
- 功能描述：将 $PC+4$ 的值写入寄存器 r_d ，然后无条件跳转。跳转的目标地址是 r_j 寄存器的值加上偏移量 $offset = sext(offsets16 \ll 2)$ 。其中， $offsets16$ 是指令码中的16位立即数。
- 汇编例子：

```
jirl r1, r7, -4    # r1 ← (PC) + 4, (PC) ← ((r7) - 4)
```

3.6 l26型指令

b

- 指令名：无条件跳转
- 使用语法：b $offset$
- 功能描述：跳转的目标地址是当前 PC 值加上偏移量 $offset = sext(offsets26 \ll 2)$ 。其中， $offsets26$ 是指令码中的26位立即数。
- 汇编例子：

```
b 0x1234    # (PC) ← (PC) + 0x1234
```

bl

- 指令名：无条件跳转并寄存器链接
- 使用语法：bl $offset$
- 功能描述：将 $PC+4$ 的值写入寄存器 r_1 ，然后无条件跳转。跳转的目标地址是当前 PC 值加上偏移量 $offset = sext(offsets26 \ll 2)$ 。其中， $offsets26$ 是指令码中的26位立即数。bl 指令在功能上类似于 jirl $r_1, offsets16$ ，但跳转范围大得多。
- 汇编例子：

```
bl -4    # (r1) ← (PC) + 4, (PC) ← (PC) - 4
```

功能部件设计

数据通路是用于描述CPU内部信息流的模型，刻画的是指令执行过程中的主要信息的基本流动路径。在图论中，一条路径通常包含若干个顶点和若干条边。因此，数据通路的设计也包含功能部件（路径的“顶点”）和部件互连（路径的“边”）两个方面。

数据通路的主要功能部件包括时钟模块、程序计数器 (Program Counter, PC)、指令存储器 (Instruction ROM, IROM)、数据存储器 (Data RAM, DRAM)、寄存器文件 (Register File, RF)和算术逻辑运算单元 (Arithmetic and Logic Unit, ALU)，如图3-1所示。

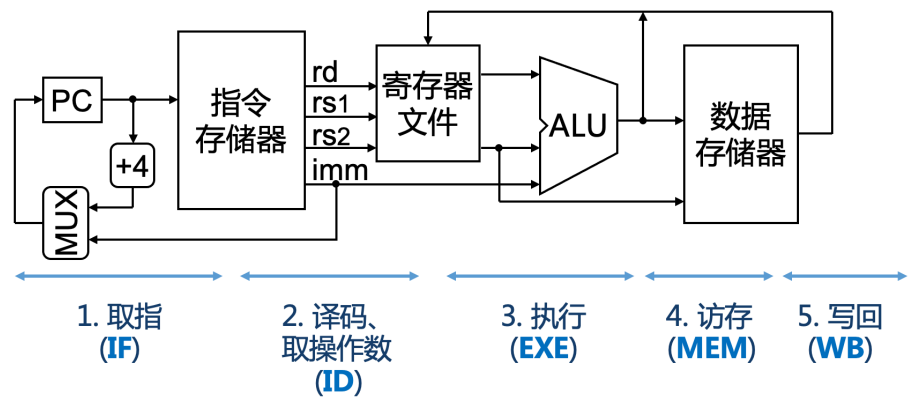


图3-1 miniRV数据通路简图

数据通路具体包含哪些功能部件取决于设计者，就像同一个数字逻辑实验可以有很多种模块划分方案。下面以表3-1所示的模块划分方案为例来说明数据通路的基本功能部件。

表3-1 功能部件列表

功能部件	功能
时钟模块	使用IP核对板上晶振时钟进行分频
PC (Program Counter)	程序指针寄存器，存储当前指令的地址
NPC (Next PC)	计算得出下一条指令的实际PC值
Inst_ROM (Instruction ROM)	指令存储器，存储指令机器码，对CPU只读
RF (Register File)	寄存器堆，存储指令的源操作数、计算结果
SEXT (Signed-Extender)	立即数扩展器，将指令码中的立即数扩展成32位源操作数

功能部件	功能
ALU (Arithmetic and Logic Unit)	完成算术运算、逻辑运算
Data_RAM (Data RAM)	数据存储器，存储主存数据，对CPU可读可写
MREQ (Memory REQuest)	通过CPU模块的访存接口向外发出读写主存的访问请求
MEXT (Memory data EXTender)	访存数据扩展器，用于对从内存读回的半字、字节数据进行扩展

1. 时钟模块

CPU需要通过系统时钟信号来控制指令执行的时序。一般地，CPU的执行速率与时钟频率成正比，即时钟频率越高，CPU的执行速度越快。然而，CPU内部的部件存在一定的物理延迟，如果时钟频率过高，这些部件来不及响应，就会产生不稳定的输出结果，从而导致指令执行结果出错。

本课程的FPGA开发板具有100MHz频率的晶振时钟源。通常单周期CPU难以在如此高的频率下运行。因此，我们需要借助Vivado的PLL时钟IP核来实现晶振时钟的分频。

1.1 时钟IP核

demo工程已经创建好了时钟IP核，同学们无需重复创建。

时钟IP核的创建方式

在Vivado中，依次点击 IP Catalog->FPGA Features and Design->Clocking，并双击 Clocking Wizard 即可创建时钟IP核，如图3-2所示。其他IP核均可通过同样的方式创建，下文不再赘述。

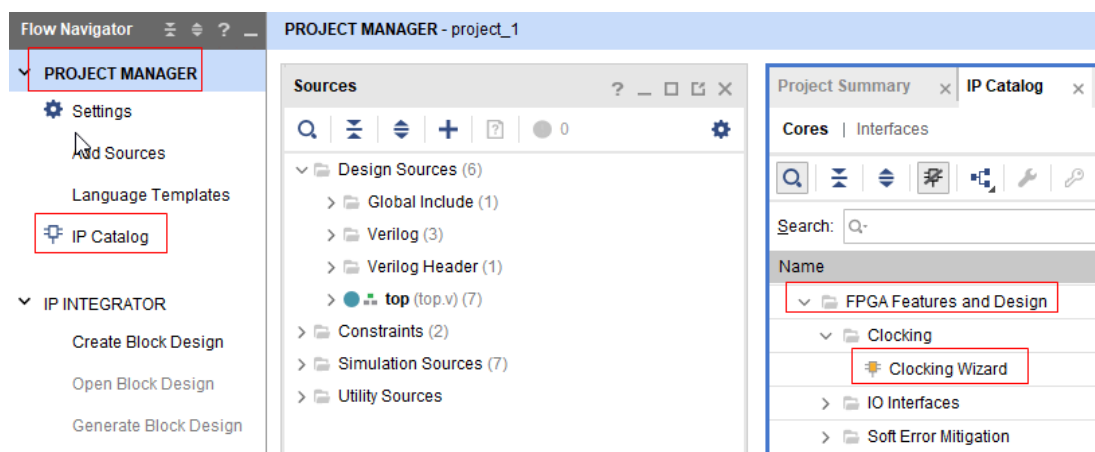


图3-2 打开 Clocking Wizard IP核

时钟IP核默认输出50MHz的时钟来驱动CPU运行。如果需要调节频率，可双击IP核打开配置窗口，点击切换到“Output Clocks”标签页，改变 `clk_out1` 的输出频率，如图3-3所示。

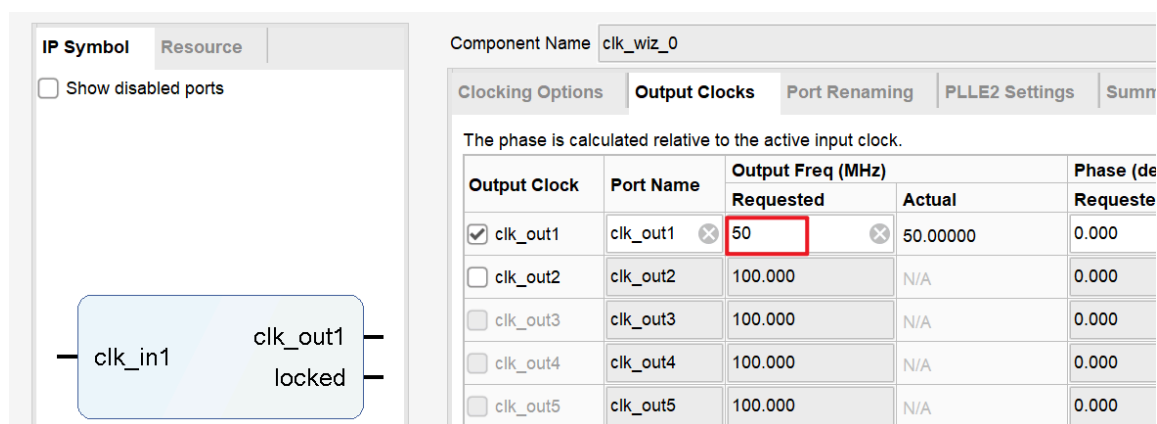


图3-3 设置时钟IP核的输出频率

修改IP核设置后，点击OK按钮。在随后弹出的对话框中，点击“Generate”按钮即可。

1.2 时钟模块的使用

demo工程的顶层模块miniRV SoC已实例化并连接好时钟IP核，相应代码如下。

miniRV_SoC.v

```
21 wire pll_clk1;
22 wire pll_lock;
23 wire sys_clk = pll_lock & pll_clk1;
```

```
24 reg sys_rst;
25 always @(posedge fpga_clk) sys_rst <= !fpga_rst | !pll_lock;
26
27 clk_wiz_0 U_clkgen (
28     .clk_in1    (fpga_clk),    // FPGA板上的晶振时钟源作为输入
29     .locked     (pll_lock),    // 初始化标志位：高电平时表示clk_out1有效
30     .clk_out1   (pll_clk1)    // 分频后的输出时钟信号
31 );
```

时钟IP核需要一定的时间进行初始化操作。初始化完成之前，输出的时钟信号是不稳定的，如图3-4所示。因此，一般需要使用 `locked` 参与生成CPU的时钟信号和复位信号，见上述代码的第23行、第25行。

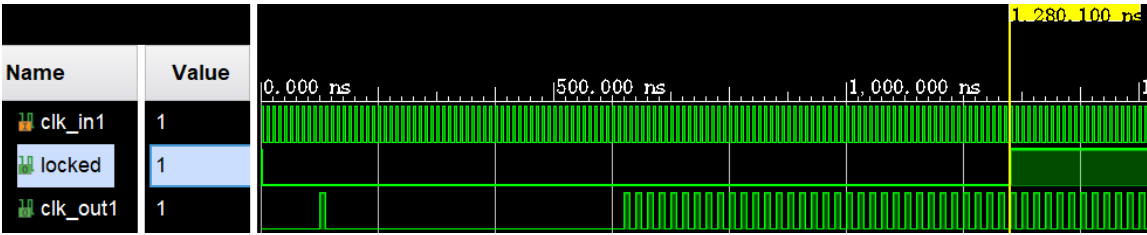


图3-4 时钟IP核的初始化波形

2. PC及NPC

PC是程序计数器，又名程序指针，物理上是一个存储着当前指令地址的32位寄存器。miniRV和miniLA都是32位的RISC架构，指令均为4字节定长，因此，PC寄存器的最低两位恒为0。

PC的初始值决定了CPU执行的首条指令的地址。因此系统复位时，必须给PC赋初始值。在demo工程中，PC的初始值定义在 `defines.vh` 头文件的 `PC_INIT_VAL`。

对于顺序执行的指令，PC值在取指完成后自动加4（PC存储的地址是字节地址，而一条指令大小是4字节）。

对于条件分支指令或直接跳转指令，CPU需要通过计算，才能得出下一条指令的PC值。因此，实现时，需要根据指令类型判断PC值如何更新，如图3-5所示。

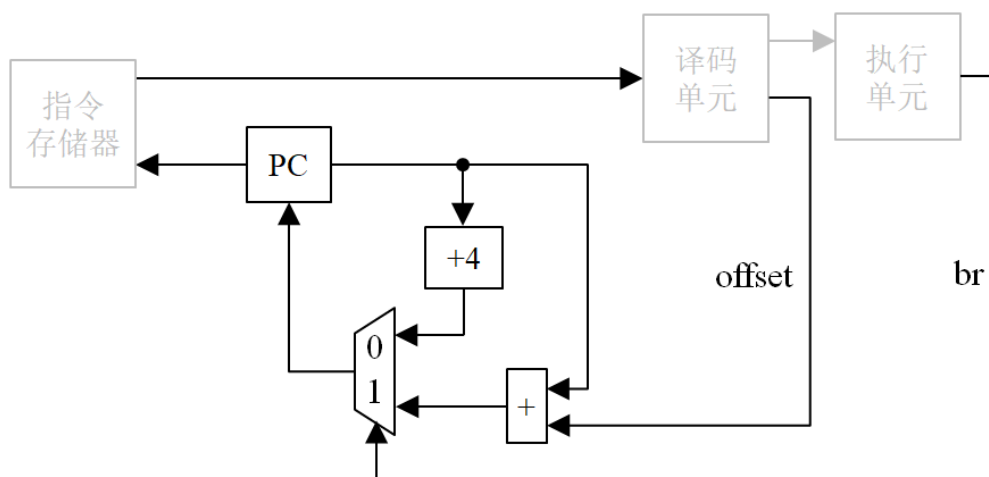


图3-5 PC原理图

在图3-5中，PC的值被当作地址输出到指令存储器。指令存储器返回指令后，由译码单元对指令进行译码，得到运算所需的源数据，并由立即数得到偏移量 `offset`。执行单元根据数据的运算结果，产生跳转标志位信号 `br`。当 `br` 为0时，PC的新值等于其旧值加4；当 `br` 为1时，PC的新值则等于PC的旧值加上跳转的偏移量 `offset`。一般地，可将产生PC新值的相关逻辑封装起来，即为demo工程的NPC模块。

3. 主存模块

demo工程已使用Block Memory IP核实现哈佛结构的指令存储器Inst_ROM和数据存储器Data_RAM。其中，指令存储器是只读的，而数据存储器是可读可写的。

Block Memory IP核的读写时序如图3-6所示。读时序的基本规律是当前时钟发出读地址，下一个时钟即可获取读数据。写时序则是在同一个时钟上升沿发出写地址、写使能和写数据即可。

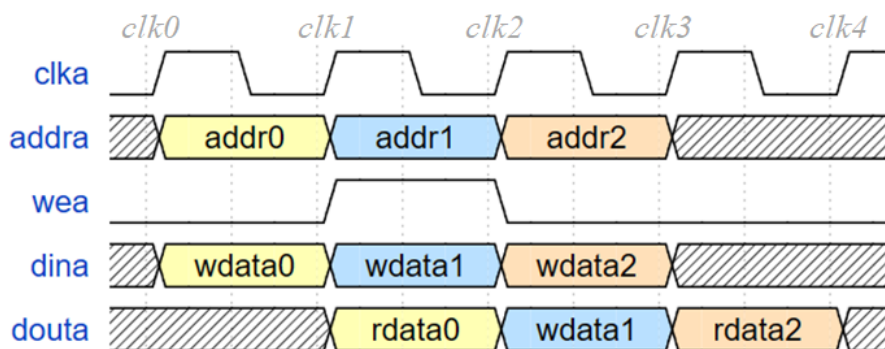


图3-6 Block Memory的读写时序

指令存储器和数据存储器的每个存储单元都是32位大小，且数据存储器的写使能信号为4bit，支持字节、半字的写操作。

3.1 Block Memory的容量修改

在Vivado中，我们可以在IP核的配置窗口修改Block Memory IP核的容量。双击实例化好的IP核，即可打开Block Memory IP核的配置窗口。点击"Port A Options"，即可设置数据深度和数据宽度，如图3-7所示。

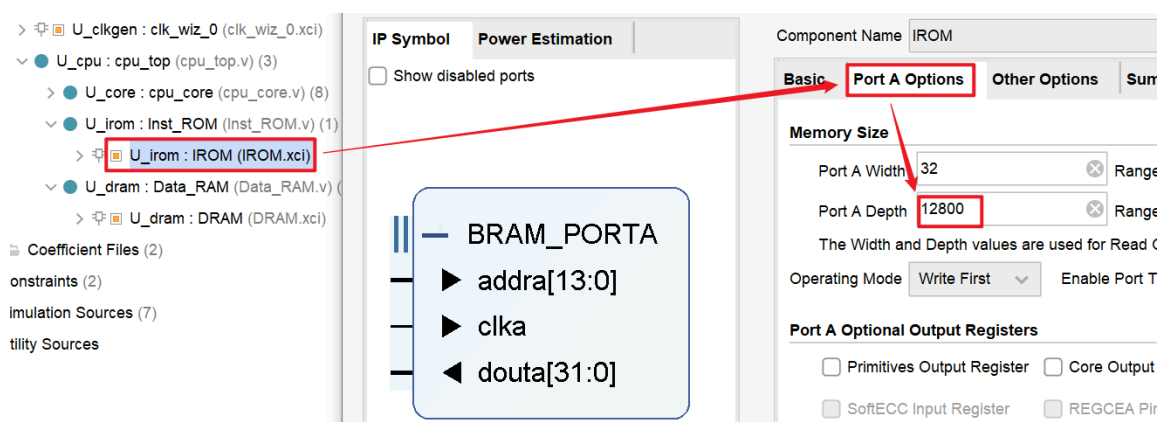


图3-7 修改Block Memory IP核的容量参数

demo工程默认配置指令存储器的大小是 $12800 \times 32\text{bit} = 50\text{KB}$ 。除非后续需要运行LLAMA2推理程序，否则不需要修改其容量。

3.2 Block Memory的初始化

在哈佛结构的计算机系统中，指令和数据分开存储。因此，当程序经过编译和汇编变成机器码之后，我们需要分别把代码段和数据段的机器码导入到指令存储器和数据存储器。

Block Memory IP核支持通过 .coe 文件导入初始数据。 .coe 文件的语法如下：

.coe文件语法（示例）

```
memory_initialization_radix = 16; ①
memory_initialization_vector = ②
3c01ffff ③
343cf000
3401ff0f
af810c04
8c020000
8c030004
00000000
.....
00000000 ④
```

- ① 表明以下数据采用16进制 (支持2、8、10、16进制)
- ② 下面每行放一个存储单元的数据, 可以不放满Block Memory IP核的配置容量
- ③ 数据之间可用逗号作为分隔符, 也可不用分隔符
- ④ 最后一行以分号作为结束符, 也可不用结束符

存储器IP核的编址方式说明

存储器IP核的地址是关于存储单元对齐的。如果存储单元大小是一个存储字，则存储器IP核的地址就是字地址。

以上述 .coe 文件为例，0地址对应第1个数据 0x3c01ffff，1地址对应第2个数据 0x343cf000，依此类推。

按照上述.coe文件的语法，把编译汇编生成的机器码拷贝到 .coe 文件中，保存并关闭，再将其拷贝到所在工程的目录下。

在Vivado中双击创建好的Block Memory IP核，在设置窗口中点击进入 Other Options 标签页，导入 .coe 文件，如图3-8所示。

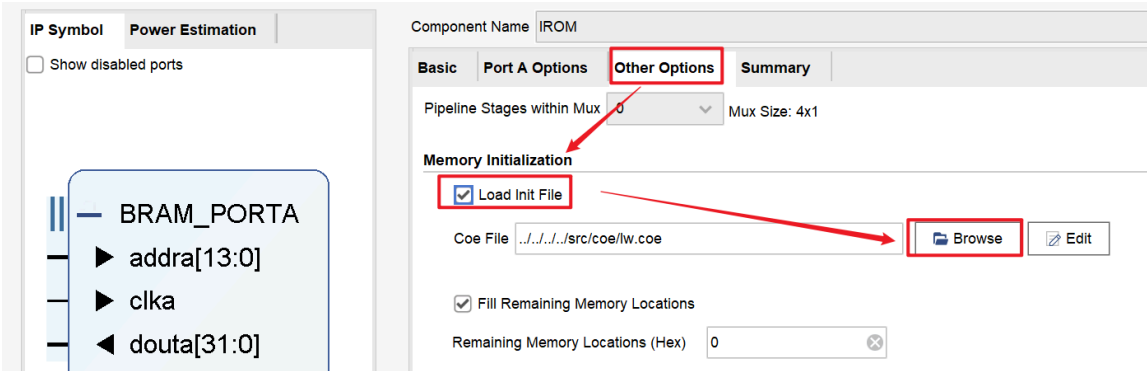


图3-8 导入.coe文件以初始化Block Memory IP核

4. 寄存器堆（RF）

miniRV和miniLA都具有32个32位寄存器，详细说明见上一节指令集介绍中的[miniRV通用寄存器](#)（或[miniLA通用寄存器](#)）。

由指令格式易知，一条指令最多需要访问3个寄存器，这决定了寄存器堆必须具有3个端口——2个读端口（对应 $rs1$ 和 $rs2$ ）和1个写端口（对应 rd ），如图3-9所示。

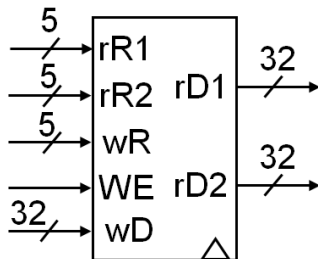


图3-9 寄存器堆模块图

一般地，寄存器读取数据采用组合逻辑，而写数据采用时序逻辑。

我们在数字逻辑设计的实验中已经设计过寄存器堆，此处不再赘述。

5. 立即数扩展器（SEXT）

miniRV和miniLA都是32位定长的RISC指令集架构，其特点是指令编码是32位的，参与运算的整数也是32位的。因此指令中的立即数位宽必然小于32，故而立即数在参与运算之前，必须先 按照指令规定的方法 被扩展成32位的数据。

立即数扩展方式通常包括 符号扩展、无符号扩展（又称零扩展）。符号扩展指的是使用立即数的符号位（即最高位）来填补空缺的高位，从而形成32位的数据。miniRV的所有立即数均采用符号扩展，而miniLA的 `andi`、`ori`、`xori` 指令采用无符号扩展，其余指令均采用符号扩展。

立即数扩展例子

某miniRV指令的16进制编码是 `0xf0038393`，对应的二进制编码是 `111100000000_00111_000_00111_001011`。查[miniRV指令速查表](#)可知该指令对应的汇编是 `addi x7, x7, -256`。根据速查表中 `addi` 指令的功能，需要对立即数 `12'b111100000000` 进行符号扩展。扩展时，使用立即数的符号位（即最高位）填充高位，因此是 `{20'hFFFFFF, 12'hF00} = 32'hFFFFFF_F00`。

6. 算术逻辑单元（ALU）

ALU接收控制器生成的控制信号，完成指令所需的算术、逻辑、移位和比较等运算。我们可以使用HDL中的运算符来实现加（+）、减（-）、按位与（&）、按位或（|）、按位异或（^）、移位（左移<<、右移>>）、比较大小等简单运算。

Verilog默认把数据当作无符号数处理。在涉及算术右移、有符号比较时，需结合 `$signed` 关键字使用。比如，`$signed(B) >>> A[4:0]` 是算术右移，`$signed(A) < $signed(B)` 是有符号比较操作。

对于 **乘除法运算，要求自行实现硬件乘/除法器**。我们在计算机组成原理实验中，已经完成了基于Booth算法的硬件补码乘法器设计。因此，只需将其集成到ALU当中即可。集成乘法器时，需要注意[乘法器模块的接口信号时序](#)。

除法器模块采用与乘法器相同的接口信号与时序，详见后文的[乘除法指令实现](#)。

7. 访存请求模块（MREQ）

demo工程的CPU采用如表3-2所示的访存接口。

表3-2 CPU的访存接口

信号	位宽	属性	功能描述

daccess_ren	4	输出	读使能信号（有效：4'hF，无效：4'h0）
daccess_addr	32	输出	读/写地址
daccess_rvalid	1	输入	数据存储器返回的读数据有效信号 有效一个时钟表示返回一个读数据
daccess_rdata	32	输入	数据存储器返回的读数据
daccess_wen	4	输出	写使能信号，支持写字、半字、字节 （有效：非零值，无效：4'h0）
daccess_wdata	32	输出	写数据
daccess_wresp	1	输入	数据存储器返回的写响应信号 有效则表示写操作已完成

为了描述方便，下面省略信号名称的 daccess_ 前缀。

访存请求模块MREQ根据访存指令的需要，对上表中的信号进行相应的赋值，从而发出读或写访存请求。

对于读访存指令，只需将 ren 置为 4'hF，并从 addr 引脚发出 字地址，然后在读回数据后由数据扩展器MEXT进行数据扩展操作即可。

对于写访存请求，则需要根据指令类型发出对应的 wen 写使能信号。指令类型、访存地址字节偏移、以及对应的写使能信号取值等如表3-3所示。

表3-3 写访存请求的信号赋值关系

写访存指令	访存地址字节偏移	wen	完成的写操作
sb（或 st.b）	2'h00	4'b0001	wdata[7:0] 写入对应存储单元的 [7:0]
sb（或 st.b）	2'b01	4'b0010	wdata[7:0] 写入对应存储单元的 [15:8]
sb（或 st.b）	2'b10	4'b0100	wdata[7:0] 写入对应存储单元的 [23:16]

写访存指令	访存地址字节偏移	wen	完成的写操作
sb（或 st.b）	2'b11	4'b1000	wdata[7:0] 写入对应存储单元的 [31:24]
sh（或 st.h）	2'b00	4'b0011	wdata[15:0] 写入对应存储单元的 [15:0]
sh（或 st.h）	2'b10	4'b1100	wdata[15:0] 写入对应存储单元的 [31:16]
sh（或 st.h）	2'b01 或 2'b11	4'h0	地址不对齐，不访存
sw（或 st.w）	2'b00	4'b1111	wdata 写入对应存储单元的 [31:0]
sw（或 st.w）	2'b01 或 2'b10 或 2'b11	4'h0	地址不对齐，不访存

demo工程的CPU仅实现了的 lw（或miniLA的 ld.w）指令，其余访存指令需要同学们按照上面的表格和说明自行完成。

8. 访存数据扩展器（MEXT）

miniRV和miniLA都是Load-Store型的指令集。所谓Load-Store型指令集，指的是指令集中只有Load和Store指令访存，其他指令不访存。miniRV和miniLA的访存指令具有3种数据访问粒度，分别是8位字节（Byte）、16位半字（Half Word）和32位字（Word）。

字节和半字的写操作，可以通过总线和存储器的4位写使能信号实现。然而，字节和半字的读操作则不能纯粹依靠读使能信号。miniRV的 lb 和 lh 指令（或miniLA ld.b 和 ld.h 指令）要求从主存取取字节和半字数据，并对所读取的数据进行符号扩展之后，才能把数据写回寄存器堆。主存储器和总线一般不支持符号扩展操作，所以CPU需要自行实现访存数据的扩展。

访存数据扩展器MEXT实现2项功能，一项是根据访存地址的字节偏移（即地址信号的最低2位）从访存读回的32位数据中选出待扩展数据（见 MEXT.v 中的 real_din 信号），另一项则是对数据进行符号扩展。

特别地，对于半字和字的访存指令，需要注意访存地址的对齐问题，即按半字访存要求访存地址最低位是 1'b0，按字访存则要求访存地址最低两位是 2'b00。CPU执行访存指令时，应当判断访存地址是否满足对齐条件。当访存地址不对齐，则CPU不应发出访存请求。

数据通路设计

在上一节“功能部件设计”中，我们为数据通路的路径设计了所需的“顶点”。现在，我们需要根据指令功能，将这些功能部件连接起来，形成路径的“边”。

设计数据通路的基本思想是：根据每条指令的功能描述和指令格式，推演出指令执行过程所需的功能部件及其输入输出信号之间的连接关系，即指令级数据通路。

在推演的过程中，还将分析功能部件需要执行的具体功能，并据此再推演相应的控制信号取值。

本课程采用表格驱动设计的方法来构建数据通路，即通过数据通路表来描述数据通路。表4-1给出了8条miniRV指令构建数据通路表的示例。

表4-1 表格驱动的数据通路设计示例

所属单元	取指单元				译码单元				执行单元		存储单元				
部件	PC		NPC		RF				ALU		MREQ		MEXT		
输入信号	npc	pc	offset	br	rR1	rR2	wR	wD	imm	A	B	ram_addr	ram_wdata	din	byte_offs
addi	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
slli	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.daccess_rdata	ALU.C[1:0]
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst[31:7][30:25][11:8]	RF.rD1	RF.rD2	-	-	-	-
bne	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst[31:7][30:25][11:8]	RF.rD1	RF.rD2	-	-	-	-
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst[31:19:12:20:30:21]	RF.rD1	SEXT.ext	-	-	-	-
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:7]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.daccess_rdata	ALU.C[1:0]

注：表中的 X.y 表示部件 X 的输出信号 y；IN.a 表示CPU的输入信号 a。



关于示例的说明

需要注意的是，表4-1所示的数据通路表仅仅是一个示例。在建模过程中，可能会面临基本功能部件不满足需求的问题，因此需要修改并完善已有基础功能部件，或者设计新的功能部件。

对所有的指令进行分析，并将数据通路表填写完整后，就得到了每一条指令执行时的数据通路。为了设计CPU，我们还需要将所有指令的数据通路综合起来。

综合的方法是：在如表4-1所示的数据通路表的最后新增一行“综合”，代表CPU的整个数据通路；然后将各个部件的输入信号全部整合到最后一行。

在数据通路表中，有些功能部件的输入信号具有2个以上的输入源，如RF的wD端口有4个输入来源。我们已经学习过数字逻辑设计的课程，可知此时需要新增一个多路选择器来满足此需求。因此，在综合数据通路表时，需要视情况添加多路选择器 (或采用其他方式处理多输入源的问题，如添加输入端口等)，如图4-1所示。

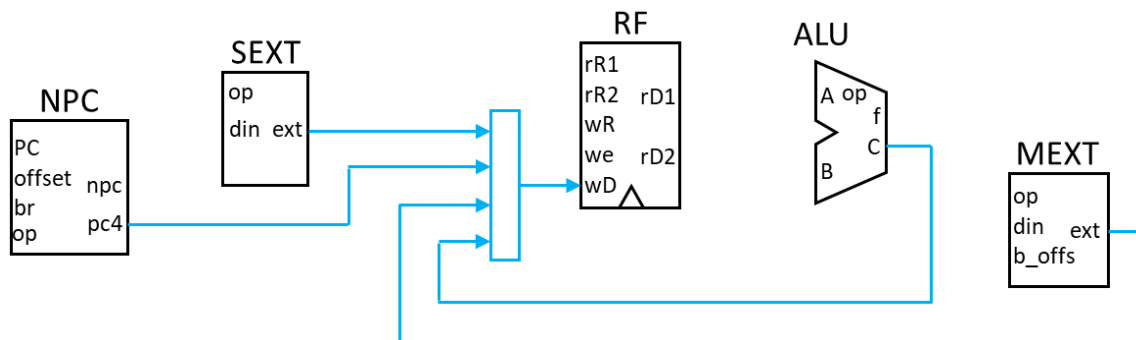


图4-1 添加多路选择器，以解决多输入源的问题

对于不同格式的立即数造成的多输入源问题，也可以使用上述方法进行综合和连接，但会造成线网资源的浪费，因此也可使用拓展输入信号位宽的方法，将指令码中可能是立即数的比特位全部输入到SEXT扩展部件当中。

最后，将综合完成的数据通路表图形化，即可构建出CPU的数据通路图。

控制单元设计

数据通路包含了完成处理器所要求的操作所必须的硬件，但数据通路要在控制器的控制下才能有条不紊地工作，才能正确地运行程序并得出程序员想要的结果。

1. 控制信号的分类

控制器的功能是根据指令的操作码 (`opcode`) 和功能码 (`funct3`、`funct7`)，产生指令执行所需的控制信号。从控制信号的功能和作用分类，可将控制信号分为 **功能选择信号**、**多路选择信号** 两种类型，如表5-1所示。

表5-1 控制信号的分类

类型	作用	控制的对象
操作选择信号	指令执行时，控制功能部件具体完成哪个操作。比如ALU对数据进行何种运算、RF要读数据还是写数据，等等	多功能部件，比如NPC、SEXT、ALU、RF等
多路选择信号	对多输入源进行选择，从而整合不同指令的数据通路	数据通路中的多路选择器

2. 确定数据通路所需的控制信号

为了设计控制器，我们需要确定数据通路需要哪些控制信号。

在上一节“数据通路设计”中，我们通过分析和综合，由指令级别的数据通路逐步构建出整个CPU的数据通路。现在，我们需要在表4-1的基础上进行扩展 —— 在“综合”的下方新增“操作选择信号”和“多路选择信号”两行；然后分析数据通路，得出每个功能部件所需的控制信号，并填入表中，如表5-2所示。

表5-2 扩展数据通路表，添加控制信号（示例）

所属单元	取指单元				译码单元				执行单元		存储单元					
部件	PC	NPC			RF				SEXT		ALU		MREQ		MEXT	
输入信号	npc	pc	offset	br	rR1	rR2	wR	wD	imm	A	B	ram_addr	ram_wdata	din	byte_offs	
addi	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-	
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-	
slli	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-	
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-	
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.daccess_rdata	ALU.C[1:0]	
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst[31:7][30:25][11:8]	RF.rD1	RF.rD2	-	-	-	-	
bne	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst[31:7][30:25][11:8]	RF.rD1	RF.rD2	-	-	-	-	
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst[31:19][12:20][30:21]	RF.rD1	SEXT.ext	-	-	-	-	
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:7]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.daccess_rdata	ALU.C[1:0]	
操作选择信号	-	-	npc_op		-	-	rf_we		-	sext_op		alu_op		ram_rop		
多路选择信号	-	-	-	-	-	-	-	rf_wsel	-	-	-	alub_sel		-	-	

接下来，把控制单元和控制信号添加到数据通路图中，即可得到形如图5-1的单周期CPU数据通路图。控制单元的输入信号是指令的操作码、功能码，输出信号则是表5-2列出的操作选择信号和多路选择信号。

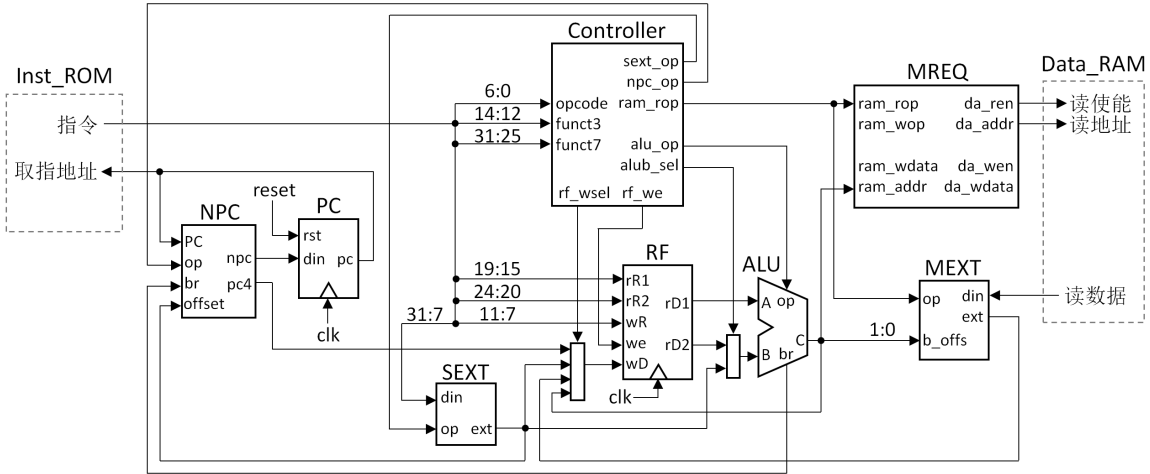


图5-1 8条指令的完整数据通路图 (示例)

3. 设计控制信号的逻辑表达式

确定了数据通路所需的控制信号之后，我们还需要确定不同指令执行时的控制信号取值，从而得出每个控制信号的真值表和逻辑表达式。

根据数据通路表及指令功能，填写形如表5-3所示的控制信号取值表。例如，`addi` 指令顺序执行，故令 `npc_op` 为 `PC4`，表示 `addi` 执行后PC的值要加4，从而实现顺序执行；此外，`addi` 指令需把加法运算结果写回到目标寄存器，因此令 `rf_we` 为 `1`，表示指令需要写回；令 `rf_wsel` 为 `WB_ALU`，表示写回的数据来源于ALU的运算结果；令 `alu_op` 为 `ADD`，表示 `addi` 指令需要ALU完成加法运算，等等。

表5-3 控制信号取值表 (示例)

指令	opcode	funct3	funct7	控制信号						
				npc_op	rf_we	rf_wsel	sext_op	alu_op	alub_sel	ram_rop
addi	0010011	000	-	PC4	1	WB_ALU	I_TYPE	ADD	SEL_EXT	0
ori	0010011	110	-	PC4	1	WB_ALU	I_TYPE	OR	SEL_EXT	0
slli	0010011	001	0000000	PC4	1	WB_ALU	I_TYPE	SLL	SEL_EXT	0
lui	0110111	-	-	PC4	1	WB_EXT	U_TYPE	-	-	0
lw	0000011	010	-	PC4	1	WB_RAM	I_TYPE	ADD	SEL_EXT	WORD_EXT
beq	1100011	000	-	BRANCH	0	-	B_TYPE	EQ	SEL_RS2	0
bne	1100011	001	-	BRANCH	0	-	B_TYPE	NE	SEL_RS2	0
jal	1101111	-	-	JUMP	0	WB_PC4	J_TYPE	-	SEL_EXT	0



保证良好的可读性

在表5-3中填写控制信号取值时，尽量采用诸如 PC4、ADD 等可读性较强的符号表达方式。实现时，只需把表格内的符号用宏定义表示即可。

确定了各个控制信号的取值之后，就可以使用卡诺图对每个控制信号的逻辑表达式进行化简，得到每个控制信号的逻辑表达式（也可使用Verilog的 **行为描述法** 来实现控制信号的生成），最终完成控制单元的设计。

多周期指令的数据通路

在本课程的单周期CPU中，大部分指令能够在一个时钟周期内完成，但有两类特殊指令：访存指令和乘除法指令。

对于访存指令而言，发出访存请求后，CPU需要等待总线访问Cache或主存，而总线和主存通常需要多个时钟周期才能完成数据的读/写操作。

对于乘除法指令而言，乘除法操作需要多个时钟周期才能完成（比如Booth补码乘法、恢复余数或加减交替的原码除法都需要N个时钟周期才能完成N位数的运算）。

下面以miniRV为例，介绍访存指令、乘除法指令的实现要点，miniLA同理，不再赘述。

demo工程实现的是单周期CPU，每条指令只会在数据通路中停留一个时钟周期，如下列代码所示。

cpu_core.v

```
107 // 按照约定的时序，ifetch_inst只在ifetch_valid有效时有效，且它们仅有效1个时钟。
108 // 此处是为了避免ifetch_valid撤销后，ifetch_inst发生变化从而导致指令执行出错。
109 assign inst = ifetch_valid ? ifetch_inst : 32'h13 /* NOP */ ;
```

指令只在数据通路中停留一个时钟周期，则指令译码产生的控制信号、寄存器操作数、立即数等信号也会只有效一个时钟周期。

为了保证访存指令和乘除法指令的正确运行，demo工程的单周期CPU在检测到当前正在执行这两类指令时，使用专门的寄存器把相应信号寄存起来。例如，对于访存指令，有下列代码：

cpu_core.v

```
147 // 遇到访存指令时，拉高ld_st_flag标志位，表示正在执行访存指令
148 assign is_ld_st = (ram_rop != `RAM_EXT_N) | (ram_wop != `RAM_WE_N);
149 always @(posedge cpu_clk or posedge cpu_rst) begin
150     if (cpu_rst) ld_st_flag <= 1'b0;
151     else if (is_ld_st) ld_st_flag <= 1'b1;
152     else if (ld_st_done) ld_st_flag <= 1'b0;
153 end
```

cpu_core.v

```
163 // 访存、乘除法指令无法在1个时钟内执行完，故先把指令的目标寄存器缓存起来
164 always @(posedge cpu_clk) begin
165     if (is_ld_st | is_mul_div) rf_wr_r <= inst[11:7];
166 end
```

cpu_core.v

```

203 // 缓存访存地址和读访存的op信号，用于在主存返回数据后，对数据进行扩展操作
204 always @(posedge cpu_clk) if (is_ld_st) alu_c_r <= alu_c;
205 always @(posedge cpu_clk) if (is_ld_st) ram_rop_r <= ram_rop;

```

当CPU的输入信号 `daccess_rvalid` 有效时，主存返回数据，此时读访存结束。因此有：

cpu_core.v

```

220 assign ld_st_done = daccess_rvalid | daccess_wresp;

```

主存返回数据后，MEXT立即通过组合逻辑对数据进行扩展，而扩展操作完成后，读访存指令（如LW）即可进行写回操作：

cpu_core.v

```

223 assign rf_we1 = ld_st_flag & daccess_rvalid | // Load指令在读
224 取到数据时写回
225 mul_div_flag & !mul_div_busy | // 乘除法指令在
226 运算完成时写回
227 ifetch_valid & rf_we & !is_ld_st & !is_mul_div; // 其他指令在取
228 到指令时写回
229
230 assign rf_wR = ld_st_flag | mul_div_flag ? rf_wR_r : inst[11:7];
231
232 always @(*) begin
233     casex ({ld_st_flag, rf_wsel})
234         {1'b0, `WB_ALU}: rf_wD = alu_c;
235         {1'b0, `WB_PC4}: rf_wD = pc4;
236         {1'b0, `WB_EXT}: rf_wD = ext;
237         {1'b1, 2'b??}: rf_wD = ram_ext; // 读访存指令的写回值是MEXT.ext
238         default : rf_wD = 32'h0;
239     endcase
240 end

```

乘除法运算的实现

实现乘除法指令时，**要求自行实现硬件乘/除法器**，不能使用运算符或现成的IP核。

我们在计算机组成原理实验中，已经完成了基于Booth算法的硬件补码乘法器设计。因此，我们可以将其集成到ALU当中。

在demo工程中，**乘/除法器的接口信号及时序完全相同**，具体请参考[乘法器的接口信号时序](#)。该时序可概括为：（1）启动信号 `start` 有效一个时钟周期表示开始运算；（2）运算过程中忙标志位 `busy` 有效；（3）当 `busy` 从有效变成无效时，运算完成并输出结果。

由上一节可知，指令机器码信号 `inst` 只有效一个时钟周期。控制器是组合逻辑，因此指令译码产生的 `alu_op` 信号也只有效一个时钟周期。ALU内部可通过组合逻辑判断 `alu_op` 的值，从而产生乘除法运算的相应标志位：

ALU.v

```
43  assign mul_flag  = 1'b0;    // TODO: 根据op的值判断当前执行的是否是mul、mulh指令
44  assign mulu_flag = 1'b0;    // TODO: 根据op的值判断当前执行的是否是mulhu指令
45  assign div_flag  = 1'b0;    // TODO: 根据op的值判断当前执行的是否是div、rem指令
46  assign divu_flag = 1'b0;    // TODO: 根据op的值判断当前执行的是否是divu、remu指令
```

显然，`mul_flag` 等标志位只有效一个时钟周期，因此这些标志位信号可用作乘/除法器的 `start` 启动信号。

类似地，当标志位有效时，把ALU的 `op` 信号寄存起来：

ALU.v

```
50  always @(posedge clk) begin
51      if (mul_flag | mulu_flag | div_flag | divu_flag)
52          op_r <= op;
53      else if (!busy)
54          op_r <= 4'h0;
55  end
```

在乘除法运算过程中，使用寄存起来的 `op_r` 信号来产生ALU的运算结果：

ALU.v

```
26  always @(*) begin
27      case (op_r != 4'h0 ? op_r : op)
28          .....
29      endcase
30  end
```

另外，还需要注意乘除法运算的符号问题。我们前面实现的是补码乘法器，而补码是用来表示有符号数的。如果使用补码乘法器来实现无符号运算，则需要在源数据的最高位额外再补一位0，从而令乘法器把数据当成正数来处理：

ALU.v

```
67 multiplier #(33) U_mulu ( // 无符号乘法器的数据位宽是33bit
68     .clk      (clk),
69     .rst      (rst),
70     .x        ({1'b0, a}),
71     .y        ({1'b0, b}),
72     .start    (mulu_flag),
73     .z        (mulu_res),
74     .busy     (mulu_busy)
75 );
```

对于除法器，此处以原码除法器为例，介绍除法运算的实现方法。无符号运算的处理方法与乘法器相同，而对于有符号运算，则需要根据符号位判断：如果是正数，则无需特殊处理；如果是负数，则需要使用“扫描法”求出其原码再进行运算：

ALU.v

```
67 divider #(32) U_div (
68     .clk      (clk),
69     .rst      (rst),
70     .x        (a[31] ? {1'b1, ~a[30:0] + 31'h1} : a),
71     .y        (b[31] ? {1'b1, ~b[30:0] + 31'h1} : b),
72     .start    (div_flag),
73     .z        (div_quo),
74     .r        (div_rem),
75     .busy     (div_busy)
76 );
```

除法器设计



除法器实现

除法器的接口信号及时序与乘法器相同，此处不再赘述。

点击下载并解压

div_test.zip

。先在

div_test

工程中实现除法器。通过仿真测试后，再把除法器模块集成到单周期CPU工程当中。

此处介绍原码除法器的设计。原码除法器的被除数、除数、商和余数均采用原码表示。

整数的原码除法将符号和数值分开运算。

令 $[x]_{\text{原}} = x_0x_1x_2 \cdots x_n$, $[y]_{\text{原}} = y_0y_1y_2 \cdots y_n$, 则有:

$$\left[\frac{x}{y}\right]_{\text{原}} = (x_0 \oplus y_0) \cdot \frac{x_1x_2 \cdots x_n}{y_1y_2 \cdots y_n}$$

其中, x_0 、 y_0 分别为 x 、 y 的符号;

$x_1x_2 \cdots x_n$ 为 x 的绝对值, 记作 x^* ;

$y_1y_2 \cdots y_n$ 为 y 的绝对值, 记作 y^* 。

由上式可知, 在原码除法运算中, 商的值由被除数和除数的绝对值相除求得, 即 $\frac{x^*}{y^*}$; 商的符号由被除数、除数的符号位经过异或运算求得, 即 $x_0 \oplus y_0$ 。余数的符号与被除数相同。

1. 恢复余数法

假设被除数、除数的 **数据位宽** 均为 n , 则基于恢复余数法的原码除法步骤如下:

Step1: 令被除数 (余数) 减去 y^* , 即加上 $[-y^*]_{\text{补}}$;

Step2: 若得到的余数为负数, 表明不够减, 故商上 0, 并加上 y^* 以恢复余数;
若得到的余数为正数, 表明够减, 故商上 1;

Step3: 若左移次数小于 n , 令被除数 (余数) 左移 1 位, 回到 **Step1**;
否则结束算法, 此时的被除数即为余数。

举个栗子 🌰

设被除数 $x = +1101B$ ，除数 $y = -0110B$ ； x 的绝对值 $x^* = 01101B$ ， y 的绝对值 $y^* = 00110B$ ， y^* 的补码 $[-y^*]_{补} = 11010B$ ，则有：

原始的恢复余数法

例：被除数 $x = +13$ ($+1101$)，除数 $y = -6$ (-0110)，求 $[x/y]_{原}$ 。

$[x]_{原} = 01101$ y 的绝对值 $[y^*] = 00110$ $[-y^*]_{补} = 11010$

① 符号位单独运算： $x_s \wedge y_s = 0 \wedge 1 = 1$ ，故商为负数；

被除数（余数）	商	说明
$\begin{array}{r} 00001101 \\ +11010 \end{array}$		$+[-y^*]_{补}$
$\begin{array}{r} 110101101 \\ +00110 \end{array}$	$---0$	余数为负，不够减，上商0 恢复余数： $+ [y^*]$
$\begin{array}{r} 000001101 \\ \boxed{000011010} \\ +11010 \end{array}$	$--0_$	恢复余数后，商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 110111010 \\ +00110 \end{array}$	$--00$	余数为负，不够减，上商0 恢复余数： $+ [y^*]$
$\begin{array}{r} 000011010 \\ \boxed{000110100} \\ +11010 \end{array}$	$_00_$	恢复余数后，商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 111010100 \\ +00110 \end{array}$	$_000$	余数为负，不够减，上商0 恢复余数： $+ [y^*]$
$\begin{array}{r} 000110100 \\ \boxed{001101000} \\ +11010 \end{array}$	$000_$	恢复余数后，商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 000001000 \\ \boxed{000010000} \\ +11010 \end{array}$	0001 $001_$	余数为正，上商1 商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 110110000 \\ +00110 \end{array}$	$\underline{0010}$	余数为负，不够减，上商0 恢复余数： $+ [y^*]$
$\begin{array}{r} 000010000 \end{array}$		

∴ 商为-0010，余数为0001。

图8-1 恢复余数法示例

简化的恢复余数法

例：被除数 $x=+13$ （+1101），除数 $y=-6$ （-0110），求 $[x/y]_{原}$.

$[x]_{原}=01101$ y 的绝对值 $[y^*]=00110$ $[-y^*]_{补}=11010$

① 符号位单独运算： $x_s \wedge y_s = 0 \wedge 1 = 1$ ，故商为负数；

被除数（余数）	商	说明
$\begin{array}{r} 0001101 \\ +11010 \end{array}$		$+[-y^*]_{补}$
$\begin{array}{r} 11011101 \\ +00110 \end{array}$	$_ _ _ 0$	余数为负，不够减，上商0 恢复余数： $+ [y^*]$
$\begin{array}{r} 00001101 \\ \boxed{00011010} \\ +11010 \end{array}$	$_ _ 0 _$	恢复余数后，商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 11101010 \\ +00110 \end{array}$	$_ _ 00$	余数为负，不够减，上商0 恢复余数： $+ [y^*]$
$\begin{array}{r} 00011010 \\ \boxed{00110100} \\ +11010 \end{array}$	$_ 00 _$	恢复余数后，商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 00000100 \\ \boxed{00001000} \\ +11010 \end{array}$	$_ 001$ $001 _$	余数为正，上商1 商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 11011000 \\ +00110 \end{array}$	$\underline{0010}$	余数为负，不够减，上商0 恢复余数： $+ [y^*]$
$\underline{00001000}$		

∴ 商为-0010，余数为0001.

图8-1 恢复余数法示例

注意，在本例子中， x 和 y 的数据位宽均为4位（最高位的符号位不算入内），故左移次数应是4。

通过观察容易发现，由于被除数的绝对值最多只有4位，故被除数（余数）首次加上 $[-y^*]_{补}$ 后，所得结果必定为负，必定需要恢复余数，即第一轮循环的有效操作仅仅是将被除数（余数）左移了一次。由此，可将恢复余数法进行简化，省去第一轮循环。

2. 加减交替法

假设被除数、除数的 数据位宽 均为 n ，则基于加减交替法的原码除法步骤如下：

Step1: 令被除数减去 y^* ，即加上 $[-y^*]_{补}$ ；

Step2: 若得到的余数为负数，表明不够减，故商上 0，令被除数左移 1 位，再加上 $[y^*]_{补}$ ；

若得到的余数为正数，表明够减，故商上 1，令被除数左移 1 位，再加上 $[-y^*]_{\text{补}}$ ；

Step3: 若左移次数已达 n 次，结束，此时的被除数即为余数；否则回到 **Step2**。

再举一个

设被除数 $x = +1101B$ ，除数 $y = -0110B$ ； x 的绝对值 $x^* = 01101B$ ， y 的绝对值 $y^* = 00110B$ ， y^* 的补码 $[-y^*]_{补} = 11010B$ ，则有：

原始的加减交替法

例：被除数 $x = +13 (+1101)$ ，除数 $y = -6 (-0110)$ ，求 $[x/y]_{原}$ 。

$[x]_{原} = 01101$ y 的绝对值 $[y^*] = 00110$ $[-y^*]_{补} = 11010$

① 符号位单独运算： $x_s \wedge y_s = 0 \wedge 1 = 1$ ，故商为负数；

被除数 (余数)	商	说明
$\begin{array}{r} 00001101 \\ +11010 \end{array}$		$+[-y^*]_{补}$
$\begin{array}{r} 110101101 \\ \underline{101011010} \\ +00110 \end{array}$	$\begin{array}{r} _ _ _ 0 \\ _ _ 0 _ \end{array}$	余数为负，不够减，上商0 商和余数左移1位 $+ [y^*]$
$\begin{array}{r} 110111010 \\ \underline{101110100} \\ +00110 \end{array}$	$\begin{array}{r} _ _ 00 \\ _ 00 _ \end{array}$	余数为负，不够减，上商0 商和余数左移1位 $+ [y^*]$
$\begin{array}{r} 111010100 \\ \underline{110101000} \\ +00110 \end{array}$	$\begin{array}{r} _ 000 \\ 000 _ \end{array}$	余数为负，不够减，上商0 商和余数左移1位 $+ [y^*]$
$\begin{array}{r} 000001000 \\ \underline{000010000} \\ +11010 \end{array}$	$\begin{array}{r} 0001 \\ 001 _ \end{array}$	余数为正，上商1 商和余数左移1位 $+ [-y^*]_{补}$
$\begin{array}{r} 110110000 \\ +00110 \\ \hline 000010000 \end{array}$	$\begin{array}{r} 0010 \\ \hline \end{array}$	余数为负，不够减，上商0 $+ [y^*]$

∴ 商为-0010，余数为0001。

图8-2 加减交替法示例

简化的加减交替法

例：被除数 $x=+13$ （+1101），除数 $y=-6$ （-0110），求 $[x/y]_{\text{原}}$ 。

$[x]_{\text{原}}=01101$ y 的绝对值 $[y^*]=00110$ $[-y^*]_{\text{补}}=11010$

① 符号位单独运算： $x_s \wedge y_s = 0 \wedge 1 = 1$ ，故商为负数；

被除数（余数）	商	说明
$\begin{array}{r} 0001101 \\ +11010 \end{array}$		$+[-y^*]_{\text{补}}$
$\begin{array}{r} 11011101 \\ \underline{10111010} \\ +00110 \end{array}$	$\begin{array}{r} _ _ _ 0 \\ _ _ 0 _ \end{array}$	余数为负，不够减，上商0 商和余数左移1位 $+ [y^*]$
$\begin{array}{r} 11101010 \\ \underline{11010100} \\ +00110 \end{array}$	$\begin{array}{r} _ _ 00 \\ _ 00 _ \end{array}$	余数为负，不够减，上商0 商和余数左移1位 $+ [y^*]$
$\begin{array}{r} 00000100 \\ \underline{00001000} \\ +11010 \end{array}$	$\begin{array}{r} _ 001 \\ 001 _ \end{array}$	余数为正，上商1 商和余数左移1位 $+ [-y^*]_{\text{补}}$
$\begin{array}{r} 11011000 \\ +00110 \end{array}$	$\underline{0010}$	余数为负，不够减，上商0 $+ [y^*]$
$\begin{array}{r} 00001000 \end{array}$		

∴ 商为-0010，余数为0001。

图8-2 加减交替法示例

同样地， x 和 y 的数据位宽均为4位，故原本需左移4次。

通过观察容易发现，由于被除数的绝对值最多只有4位，故首次循环时， $[-y^*]_{\text{补}}$ 总是与被除数（余数）高位的 $4'b0000$ 相加，然后随着被除数（余数）左移1位，接着再加上 $[y^*]_{\text{补}}$ 。这一系列操作可以简化为将 $[-y^*]_{\text{补}}$ 直接与左移1位的被除数相加，原理如下：

$$\begin{aligned} \{[-y^*]_{\text{补}}, x^*\} \ll 1 + \{[y^*]_{\text{补}}, 4'b0000\} &= 2^{4+1} \cdot [-y^*]_{\text{补}} + 2^1 \cdot x^* + 2^4 \cdot [y^*]_{\text{补}} \\ &= 2^4 \cdot (2 \cdot [-y^*]_{\text{补}} + [y^*]_{\text{补}}) + 2^1 \cdot x^* \\ &= 2^4 \cdot [-y^*]_{\text{补}} + 2^1 \cdot x^* \\ &= \{[-y^*]_{\text{补}}, 4'b0000\} + (x^* \ll 1) \end{aligned}$$

3. 除法器参考架构

分析原理可知，原码除法包含求补、加法、左移、异或操作，在运算过程中还需判断被除数（余数）的符号从而选择不同的操作，并在循环一定次数后结束运算。

求补、加法、左移、异或操作很容易通过组合逻辑实现；符号判断可通过特定的控制逻辑实现；而循环次数的控制则可通过计数器实现。

此处给出原码除法器的参考实现架构，如图8-3所示。

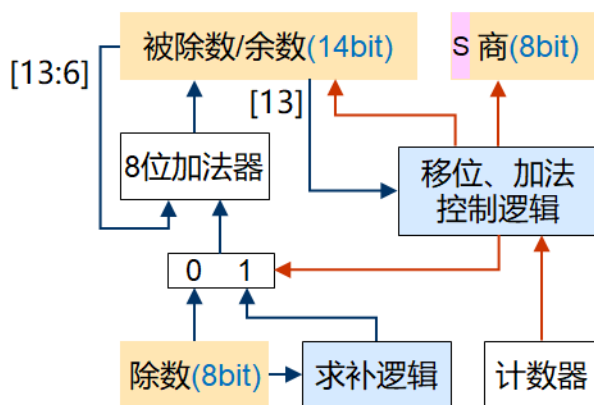


图8-3 原码除法器参考架构

在图8-3中，深蓝色箭头表示数据，深橙色箭头则表示控制信号。

控制逻辑接受2个输入：一个是被除数（余数）的符号位，另一个则是计数器的计时结束信号。控制逻辑产生3个控制信号：一个控制被除数（余数）需要与 $[y^*]_{\text{补}}$ 还是与 $[-y^*]_{\text{补}}$ 相加，即控制图8-3中的多路选择器；另外一个控制被除数（余数）的左移；还有一个控制上商。

复位时，各寄存器均被清零，计数器被复位。运算时，控制逻辑通过判断被除数（余数）最高位的符号位，控制上商以及加法器的数据源选择；加法器将被除数（余数）的高8位与 $[y^*]_{\text{补}}$ 或 $[-y^*]_{\text{补}}$ 相加，结果更新到被除数（余数）；此外，控制逻辑接收计数器的计时信号，控制被除数（余数）的移位操作，并在移位6次后结束除法运算。

单周期CPU时序

1. 单周期指令的时序

在模板工程中，单周期CPU的取指逻辑是：（1）复位信号撤销时，通过检测复位信号下降沿发出首次取指；（2）一条指令执行完毕之后，发出下一条指令的取指请求，如下列代码所示。

cpu_core.v

```
85 // 复位信号发生边沿变化时首次取指；当前指令执行完毕后取下一条指令
86 assign ifetch_req = first_req | inst_finished_r;
87 assign ifetch_addr = pc;
```

单周期指令，指的是取到指令机器码之后，能够在时钟周期之内执行完毕的指令，比如加法指令、分支跳转指令等。这些指令的时序如图9-1所示。

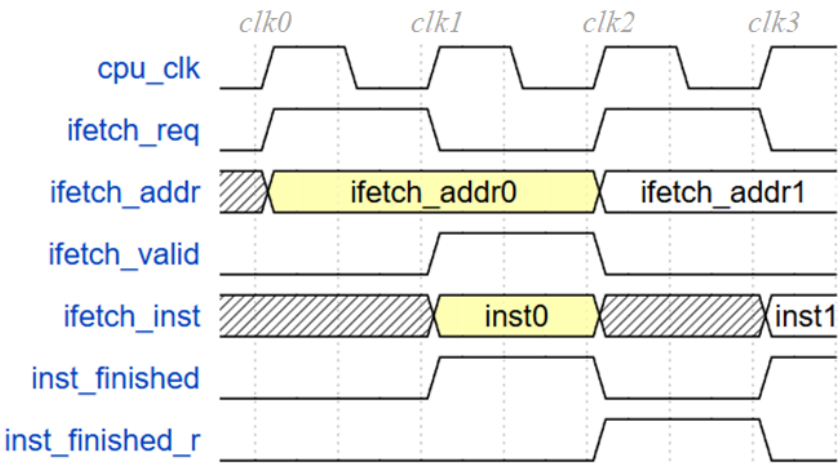


图9-1 单周期指令时序图

时序解读 🧐

- 【clk0】 ifetch_req 有效，表示此刻CPU发出了某条指令的取指请求。
- 【clk1】 ifetch_valid 有效，表示指令存储器返回了上一次取指请求所对应的指令机器码。
假设该指令是单周期指令，则指令在当前的时钟周期执行完毕，因此 **指令执行完毕的标志位信号 inst_finished 有效**。
- 【clk2】 inst_finished_r 是 inst_finished 经过CPU时钟打一拍之后得到的信号。根据上文所述的取指逻辑，
此时 inst_finished_r 有效，故CPU发出下一条指令的取指请求。

2. 多周期指令的时序

多周期指令，指的是取到指令机器码之后，不能在一个时钟周期之内执行完毕的指令，比如访存指令、乘除法指令。

访存指令的时序如图9-2所示。

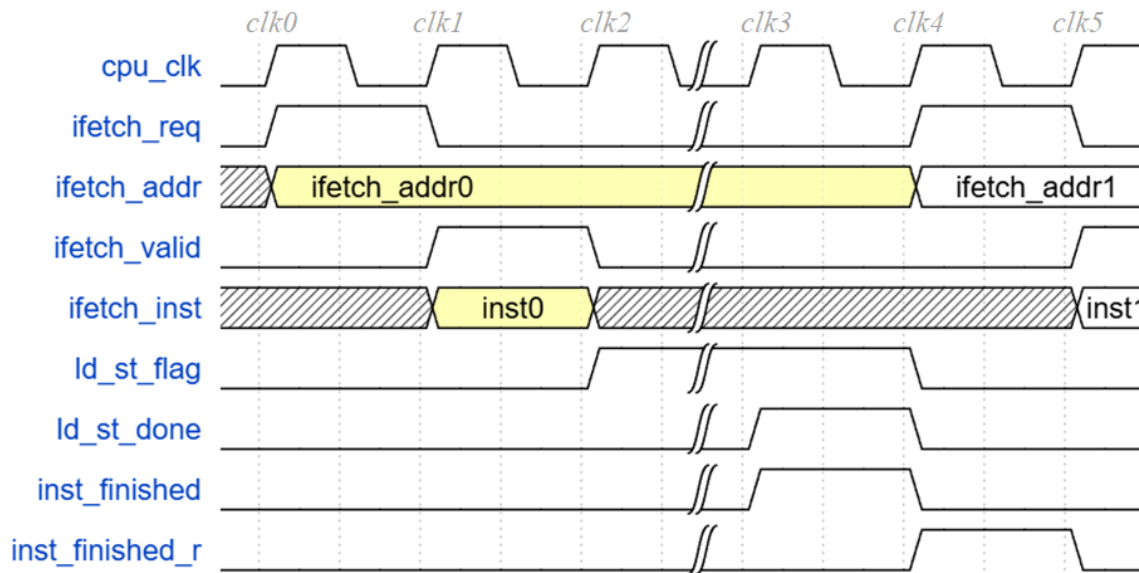


图9-2 多周期指令时序图

时序解读 🧐

- **【clk0】** `ifetch_req` 有效，表示此刻CPU发出了某条指令的取指请求。
- **【clk1】** `ifetch_valid` 有效，表示指令已取到。此时，CPU通过译码，检测到当前指令是访存指令。
- **【clk2】** CPU发出访存请求，同时访存标志位信号 `ld_st_flag` 有效。
- **【clk3】** 若干个时钟后，访存操作完成，此时 `ld_st_done` 信号有效。CPU检测到 `ld_st_flag` 和 `ld_st_done` 同时有效时，指令执行完毕，故 `inst_finished` 有效。

乘除法指令的时序与访存指令类似，如图9-3所示。

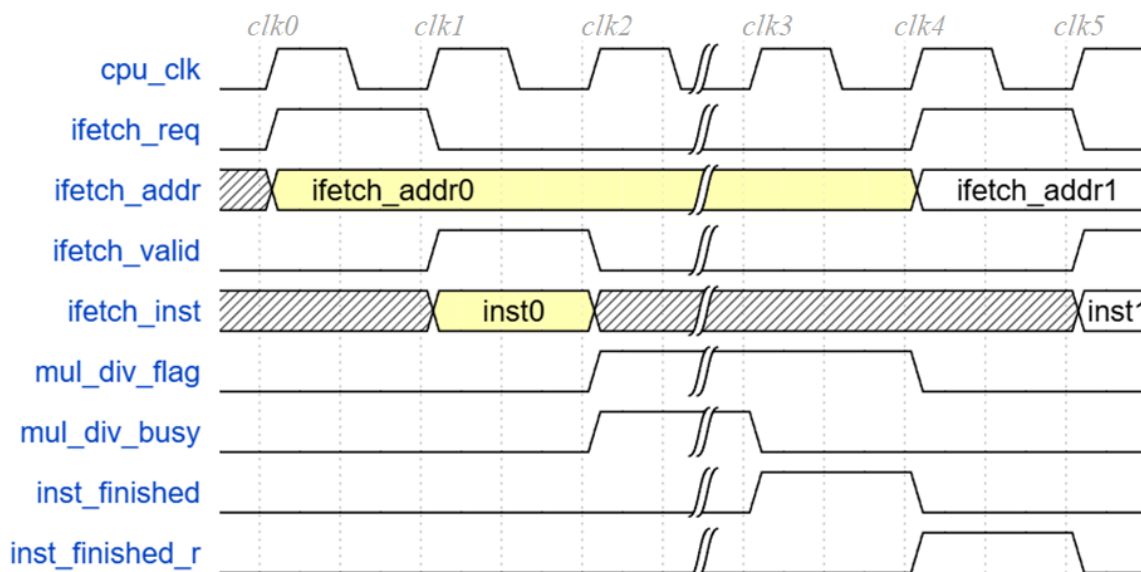


图9-3 多周期指令时序图

时序解读 🧑🏻‍🔬

- **【clk0】** `ifetch_req` 有效，表示此刻CPU发出了某条指令的取指请求。
- **【clk1】** `ifetch_valid` 有效，表示指令已取到。此时，CPU通过译码，检测到该指令是乘除法指令，乘除法器的启动信号 `start` 被拉高。
- **【clk2】** 乘除法标志位信号 `mul_div_flag` 有效，同时 `mul_div_busy` 信号被乘除法器拉高。
- **【clk3】** 若干个时钟后，乘除法运算完成，此时 `mul_div_busy` 从高电平变成低电平。CPU检测到 `mul_div_flag` 有效且 `mul_div_busy` 无效时，指令执行完毕，故 `inst_finished` 有效。

CPU功能验证原理

在数字逻辑设计的实验课程中，我们已经学习了如何使用时序仿真来对数字电路进行功能验证。所谓仿真，指的是基于软件模拟 (而非电路实测)的方法来验证电路功能，其基本原理如图10-1所示。

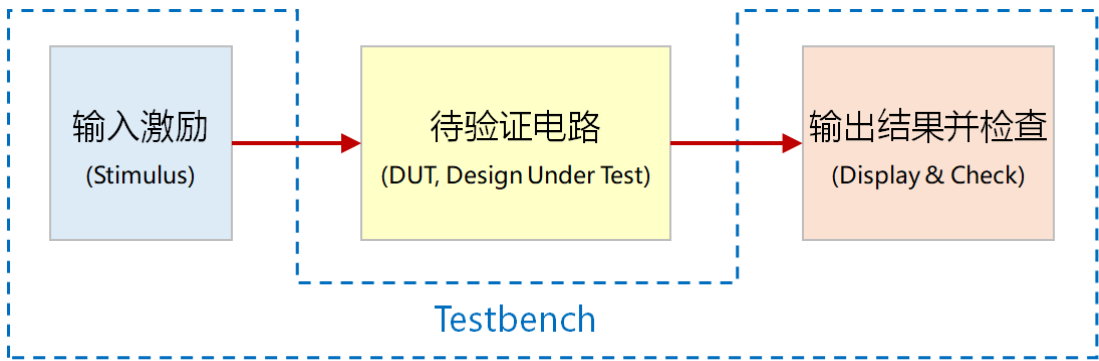


图10-1 仿真原理

在图10-1中，我们给待验证电路 (DUT, Design Under Test) 添加一些特定的输入激励，然后观察DUT的输出结果是否符合我们的预期。其中，输入激励和输出结果的检查合称为Testbench。

CPU也是数字电路，也可以采用时序仿真的方法对其进行功能验证。但CPU具有一定的复杂性，采用时序仿真验证的效率较低。一方面，CPU的顶层模块通常只包含时钟、复位以及和存储芯片及外设进行交互的信号。仅通过对顶层模块进行验证，验证结果一旦出错，难以定位错误点。因此，只能分模块进行验证，或者借助外设输出相应的调试信息。另一方面，出错点很有可能在通过测试程序的逻辑路径传递很远之后，才能被发现。为了提高验证效率，可以将图10-1中的输入激励从物理信号更换成一段特定的指令序列，并通过观察程序执行结果来验证CPU功能。这种方法虽然大幅提高了验证效率，但无法解决难以定位错误点的问题。

一种更好的验证方法是使用Trace比对，这也是本课程所使用的方法。所谓Trace，是指CPU执行指令序列时产生的信息 (包括PC和写寄存器的信息等)。

Trace比对的基本原理如图10-2所示。

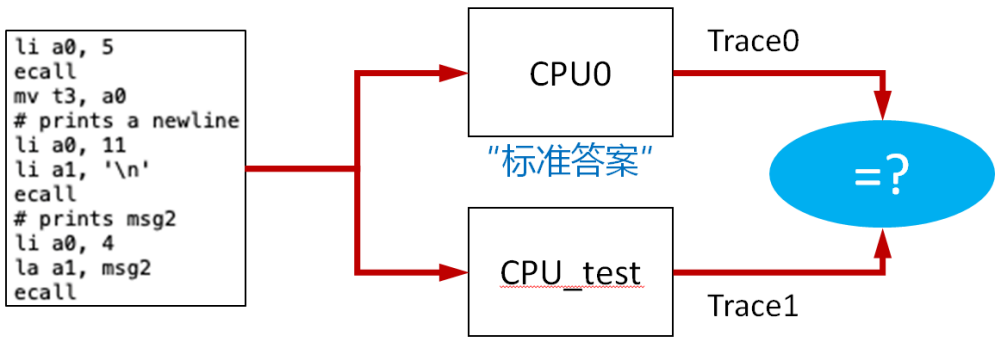


图10-2 Trace比对的基本原理

使用Trace比对进行CPU功能验证的基本方法是：

- (1) 用已知功能正确的CPU运行测试程序，记录Trace0 (Golden Trace)；
- (2) 用待验证CPU运行相同的测试程序，产生Trace1；
- (3) 将Trace1和Trace0进行实时比对，如果出现不同，立即报错并停止。

关于Trace测试框架的详细介绍及使用说明，请查看[Trace测试说明](#)章节。

单周期CPU性能分析

CPU内部既包含时序逻辑部件 (如PC、RF、DRAM), 也包含组合逻辑部件 (如IROM、控制单元、ALU等)。

对于时序逻辑电路, 输入信号需要在时钟信号的有效边沿处满足相应的建立时间和保持时间的要求, 而输出信号则相比时钟有效边沿存在一定的输出延迟。

对于组合逻辑电路, 当输入端信号发生改变, 经过一定的逻辑门电路的延时, 输出端就会发生相应的变化, 并且一直维持直到输入端再次发生变化。

指令执行时, 存在寄存器输出延迟、IROM读延迟、控制单元输出延迟、ALU运算延迟、寄存器写延迟等, 这些延迟统称为指令的执行延迟。

单周期CPU执行的大部分指令都是单周期指令。这些指令的执行延迟全部都落在同一个时钟周期之内。比如ADD指令的执行延迟及时序如图11-1所示。

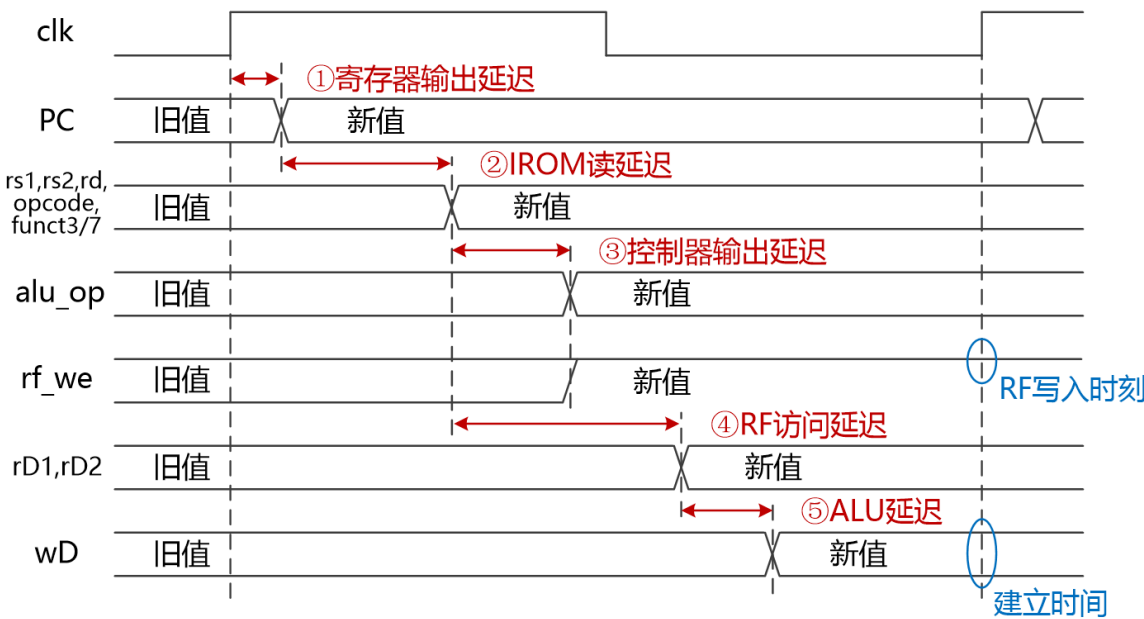


图11-1 ADD指令执行延迟示意图

在图11-1中, 所有红色箭头加起来代表了ADD指令的执行延迟。显然, 只有 **当指令的执行延迟不超过一个时钟周期的时间, 指令才能产生正确的执行结果**。换言之, **对于单周期CPU而言, CPU的时钟频率受限于执行最慢的单周期指令**。

实验步骤

(1) 从首页的课程材料下载链接，下载数据通路表和控制信号取值表的模板，并根据指令集和上课的实验室，下载对应的Vivado模板工程，如图12-1所示；



图12-1 根据实际情况，选择其中一个demo工程

- (2) 结合数据通路表和控制信号表示例，运行仿真，熟悉模板工程和示例代码；
- (3) 小组成员各自完成A、B组指令的数据通路表和控制信号表；
- (4) 与组员讨论协作，对数据通路进行整合，画出单周期CPU的完整数据通路图；
- (5) 小组成员各自根据A、B组指令的数据通路表和控制信号表，划分子模块，为每个模块新建 .v 源文件，并使用Verilog HDL实现其内部逻辑；
- (6) 小组成员各自实现A、B组指令的单周期CPU，并使用Basic Trace对各自实现的CPU进行调试和验证；
- (7) 与组员讨论协作，整合单周期CPU的实现代码，使用Basic Trace对完整的单周期CPU进行测试和验证。

A. Verilog宏定义的使用

不管是使用高级语言进行软件开发，还是使用HDL进行硬件设计，都要尽量遵循代码可配置的原则。对于常量或参数，应当在代码中使用宏定义的方式来引用。使用宏定义，不仅可以保证代码具有良好的可配置性，还能提高代码的规范性和可读性。

我们在“控制单元设计”的一节中，设计了如表5-2所示的控制信号取值表。应该注意到，明明是控制信号的取值表，表中的npc_op和alu_op却用字符串作为信号取值，显得“格格不入”。事实上，npc_op和alu_op的取值就是以宏定义的方式来表示的。将npc_op、alu_op与其他控制信号相比，可以发现宏定义的表达方式不仅更方便阅读、更容易为人所理解，而且有利于代码维护。

Verilog HDL中使用宏定义的方法是：

- (1) 新建 .vh 头文件 (如 param.vh 或 defines.vh)，用来编写宏定义；
- (2) 使用 ``define` 关键字定义宏参数，如图10-1所示。

```
param.vh

1  `ifndef PARAM_VH
2  `define PARAM_VH
3
4  // syntax: `define <macro name> <parameter>
5  `define ADD      3'b000
6  `define SUB      3'b001
7  `define OR       3'b010
8  `define AND      3'b011
9  .....
10
11 `define STATE_IDLE 4'b0001
12 `define STATE_WRIT 4'b0010
13 `define STATE_WORK 4'b0100
14 `define STATE_RETU 4'b1000
15 .....
16
17 `endif
```

图10-1 Verilog HDL定义宏参数（示例）

(3) 在需要使用宏定义的设计文件中，使用 ``include` 语句包含头文件，并通过 ``` 符号使用宏定义，如图10-2所示。

```
1  `include "param.vh"           // 包含头文件
2
3  module SOME_MODULE (
4      input wire      cpu_clk,
5      input wire      cpu_rst,
```

```
6     output reg [ 2:0] out
7 );
8
9     always @ (posedge cpu_clk or posedge cpu_rst) begin
10         if (rst) begin
11             out <= `ADD;    // 使用宏定义
12         end else begin
13             .....
14         end
15     end
16
17 endmodule
```

图10-2 包含头文件并使用宏定义

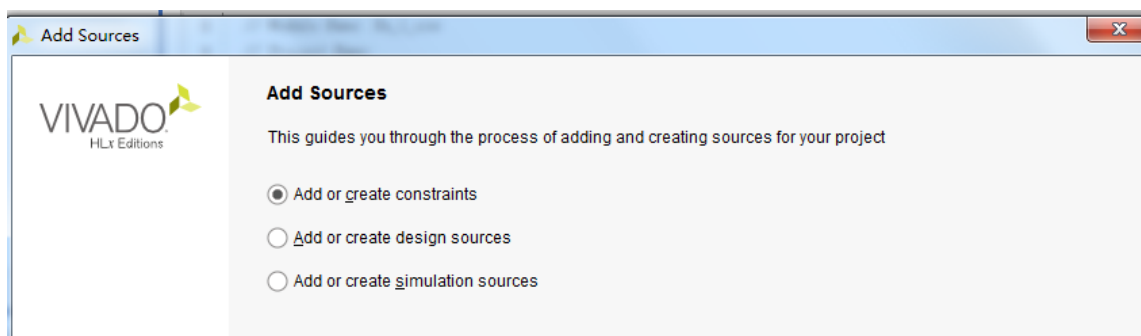
B. 引脚约束文件

下板前，需为顶层模块编写引脚约束文件 `.xdc`，从而把顶层模块的接口信号与FPGA芯片的引脚连接起来。

模板工程已提供复位、时钟以及基本外设的引脚约束。一般情况下不需修改约束文件。

1. 添加/创建约束文件

在Vivado中，右键点击 `Project Manager` 下面 `Sources` 中的 `Constraints`，在弹出的菜单中选择 `Add Source...`，并按照图B-1所示的设置点击 `Next`。



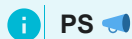
图B-1 添加或创建约束文件

2. I/O引脚约束

约束文件初始为空，其基本语法如下列代码所示。

```
1 # 语法格式为set_property PACKAGE_PIN (pin location) [get_ports (port name)]
2 set_property PACKAGE_PIN P5 [get_ports sw_i[0]]
3 set_property PACKAGE_PIN F6 [get_ports led_o[0]]
4
5 # 语法格式为set_property IOSTANDARD (level:LVDS,LVCMOS18,LVCMOS33 etc.) [get_ports
6 (port name)]
7 set_property IOSTANDARD LVCMOS33 [get_ports sw_i[0]]
8 set_property IOSTANDARD LVCMOS33 [get_ports led_o[0]]
```

约束文件主要包含两部分内容，第一部分是信号和FPGA芯片引脚的对应关系，比如第1行是将FPGA芯片的 `P5` 引脚和信号 `sw_i[0]` 信号绑定。第二部分是引脚的电气属性设置，`IOSTANDARD` 全部设置成 `LVCMOS33`，表示该IO电平标准是 `CMOS 3.3v`（即高电平对应3.3V，低电平对应0.0V）。



引脚约束代码也可写成如下列代码所示的语句。

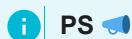
```
1 set_property -dict { IOSTANDARD LVCMOS33 PACKAGE_PIN P5 } [get_ports sw_i[0]]
2 set_property -dict { IOSTANDARD LVCMOS33 PACKAGE_PIN F6 } [get_ports led_o[0]]
```

引脚分配既可以参考[开发板使用须知](#)，也可以直接在开发板的丝印上读出。例如，观察开发板最右侧的拨码开关下方丝印着“SW0-W4”，表示拨码开关SW0对应FPGA的W4引脚。

3. 时钟约束

约束文件除了对物理引脚进行约束，还可以对时序进行约束。感兴趣的同学们可以阅读参考资料《[约束功能概述](#)》。

如果Vivado工程需要使用时钟信号，为了方便后续时序分析，以及为了让工具能够根据给定的频率进行逻辑优化，则需要添加始终约束文件。



在工程中，除了顶层模块I/O引脚的约束文件外，还可以新建一个始终约束文件 `clock.xdc`，在其中添加如下列代码所示的约束语句：

clock.xdc

```
1 # 语法格式为create_clock -name (clock name) -period (time) [get_ports (port name)]
2 create_clock -name fpga_clk -period 10 [get_ports fpga_clk]
```

其中，`-period` 项表示时钟信号的周期大小（单位：ns）。比如Minisys的板上时钟为100MHz，则时钟周期为10ns，因此是 `-period 10`。